เส้นทางผู้ใช้ 1 รอบ: สมัครสมาชิก → ตรวจอาการ → ประวัติ → ให้คะแนน → แดชบอร์ด (ช่วงที่ 1–9)

โปรเจกต์: appliance-repair-assistant (backend Express + MySQL, frontend React + Vite) **ใช้อ่านยังไง:** เดินตามผู้ใช้ชื่อ "สมชาย" ทีละขั้น แต่ละขั้นบอกว่าผ่าน **ไฟล์ไหน** → **function ไหน** → **บรรทัดไหน** ท้ายแต่ละช่วงมีข้อสังเกตและเฉลยคำถาม

> **สถานะการตรวจเลขบรรทัด (ตรวจเมื่อ 25 ต.ค. 2569)**
>
> - ช่วง 1, 2, 8, 9: เทียบกับไฟล์บนดิสก์แล้ว
> - ช่วง 3, 4, 5: กู้จากข้อความที่ตอบในแชต ยังไม่ได้เทียบเลขบรรทัดใหม่ ช่วง 5 ยัง mark ว่าต้องอ่านซ้ำก่อนสอบ
> - ช่วง 6, 7: เขียนไว้เมื่อ 24 ต.ค. เทียบบางส่วน ยังไม่ได้เทียบใหม่ทั้งหมด
> - เลขบรรทัดที่เลื่อนอาจต้องเปิดไฟล์เทียบอีกครั้ง (ไฟล์โค้ดถูกลบคอมเมนต์ทำให้เลขเปลี่ยน)
> - ทุกอย่างมาจากการอ่านโค้ด ไม่ได้รันเว็บหรือยิง API จริง

สารบัญ

1. สมัครสมาชิก
2. เข้าสู่ระบบ + JWT + auth middleware
3. เลือกเครื่อง → เลือกรุ่น
4. พิมพ์อาการ → ค้นหาด้วย embedding → เริ่มตรวจ
5. เดินผังทีละขั้น
6. บันทึกประวัติ
7. หน้าประวัติการซ่อม
8. ให้คะแนน 👍/👎 + ข้อความ
9. แดชบอร์ดผู้ดูแล

ช่วงที่ 1: สมัครสมาชิก (Register)

ภาพรวม: สมชาย (guest) กดลิงก์ "สมัครสมาชิก" กรอกชื่อ อีเมล รหัสผ่าน แล้วกดปุ่ม หน้าจอตรวจเบื้องต้นก่อน จากนั้นยิง `POST /auth/register` backend ตรวจซ้ำ แปลงรหัสผ่านเป็น hash บันทึกแถวใหม่ในตาราง `users` และออก JWT (บัตรผ่าน) กลับมา หน้าจอเก็บบัตรไว้ ทำให้สมชายล็อกอินทันทีหลังสมัคร การสมัครเป็นแค่ทางเลือกเสริม ไม่สมัครก็ตรวจอาการได้ตามปกติ

ก. ฝั่งหน้าจอ: จากการกดลิงก์ถึงการยิง API

1. **ไปหน้าสมัคร:** ลิงก์ในเมนูซ้ายคือ `NavLink to="/register"` (`AppShell.tsx:74`) และ `App.tsx:49` จับคู่ `/register` กับ `<RegisterPage />`
2. **หน้าเก็บค่าที่พิมพ์:** `RegisterPage.tsx:20-23` มี state 4 ตัว ได้แก่ `email`, `password`, `displayName` และ `fieldErrors` (ข้อความผิดพลาดใต้ช่อง)
3. **ปุ่มสมัคร:** `<form onSubmit={handleSubmit}>` อยู่ที่ `:49` และปุ่ม `type="submit"` อยู่ที่ `:90-92` กดแล้วเรียก `handleSubmit`
4. **ตรวจเบื้องต้นก่อนส่ง:** `handleSubmit` (`:34-40`)

```
event.preventDefault()
const errors = validateRegister(email, password, displayName)
setFieldErrors(errors)
if (hasErrors(errors)) return   // ไม่ผ่าน → ไม่ยิง API
submit.mutate()
```

- `validateRegister` อยู่ที่ `lib/authForm.ts:35-53`
- อีเมลต้องมีรูปแบบถูก (`:38`)
- รหัสผ่านต้องยาว 8–72 ตัวอักษร (`:41`) **และ** ไม่เกิน 72 ไบต์ (`:43`) เพราะ bcrypt อ่านแค่ 72 ไบต์แรก อักษรไทย 1 ตัวกินราว 3 ไบต์
- ชื่อที่แสดงต้องยาว 1–100 หลังตัดช่องว่าง (`:47-50`)

5. **สั่งส่ง:** `submit` (`RegisterPage.tsx:25-28`) คือ `useMutation` ที่เรียก `register(email.trim(), password, displayName.trim())` และเมื่อสำเร็จจะ `navigate(next ?? '/history', { replace: true })` `useMutation` เป็น "การกระทำที่เปลี่ยนข้อมูล" ต่างจาก `useQuery` ที่ดึงข้อมูลเอง มันทำงานเมื่อสั่ง `.mutate()` เท่านั้น
6. **provider รับช่วง:** `register` ใน `AuthProvider.tsx:93-98` เรียก `applySession(await apiRegister(email, password, displayName))`
7. **api client:** `api/auth.ts:26-33`

```
request<AuthResponse>('/auth/register', {
  method: 'POST',
  body: { email, password, displayName },
  anonymous: true,
})
```

`anonymous: true` แปลว่าไม่แนบ token เพราะผู้สมัครยังไม่มี 8. **ประกอบและยิง fetch:** `request()` ใน `api/http.ts:39-86`

- ต่อ URL = `VITE_API_URL` + `/auth/register` (`:41`)
- เพราะ `anonymous` จึงไม่หยิบ token (`:43`)
- ใส่ `Content-Type: application/json` เพราะมี body (`:46`)
- ส่งด้วย `fetch` (`:53-57`)
- ถ้าเครือข่ายล่ม จะโยน `NETWORK_ERROR` (`:58-60`)

ข. ฝั่งเซิร์ฟเวอร์: จากรับคำขอถึงตอบกลับ

1. **ผ่านด่านแรก:** `app.ts:66` คือ `cors(...)` (อนุญาตให้ frontend คนละโดเมนเรียกได้) แล้ว `:67` คือ `express.json()` ที่แปลง body เป็น `request.body`
2. **เข้า router ตาม path:** `app.ts:71` คือ `app.use('/auth', authRoutes(...))` แล้ว `auth.routes.ts:11-13`

```
router.post('/register', asyncRoute(async (request, response) => {
  response.status(201).json(await auth.register(parseRegisterBody(request.body)));
}));
```

- ไม่มี auth middleware เพราะคนสมัครยังไม่มี token
- ตอบ **201** (สร้างของใหม่) และ `asyncRoute` ส่ง error ของ handler แบบ async ไปที่ error handler กลาง (`error-handler.ts:111`)

3. **ตรวจ input ซ้ำ:** `parseRegisterBody` (`auth.dto.ts:89-96`) หยิบเฉพาะ `email`, `password`, `displayName` ฟิลด์อื่นอย่าง `role` ถูกเมิน จึงไม่มีทางส่งให้ service ใช้
   - `parseEmail` (`:53-62`): ตัดช่องว่าง แปลงเป็นตัวพิมพ์เล็ก (`:54`) ตรวจความยาวและรูปแบบ
   - `parsePassword` (`:65-78`): ตรวจ 8–72 ตัวอักษร และตรวจไบต์ด้วย `Buffer.byteLength` (`:72`)
   - `parseDisplayName` (`:80-86`): ตัดช่องว่าง ตรวจ 1–100
   - ผิดอย่างใดอย่างหนึ่ง → `ApiError 400 INVALID_AUTH_REQUEST`
4. **ตรรกะสมัคร:** `AuthService.register` (`auth.service.ts:33-53`)
   - `hash(body.password, BCRYPT_ROUNDS)` (`:34`, rounds = 10 ที่ `:10`) แปลงรหัสผ่านเป็น hash เก็บรหัสจริงไม่ได้
   - สร้างผู้ใช้ผ่าน `usersStore.create` (`:38-44`) ด้วย id แบบ `randomUUID()` และ **`role: Role.User` ตายตัว** (`:43`) จึงสมัครเป็น admin ผ่านหน้านี้ไม่ได้ (admin สร้างได้ทางเดียวคือ `scripts/seed-admin.js` ตามคอมเมนต์ใน `001_users.sql:11`)

- ถ้าอีเมลซ้ำ (`DuplicateEmailError`) แปลงเป็น `emailTaken()` ที่ `:46-47` ซึ่งตอบ 409
- ออกบัตรด้วย `this.issue(user)` (`:52`)

5. **บันทึกลง DB:** `UsersStore.create` (`users.store.ts:65-84`)

```
INSERT INTO users (id, email, password_hash, display_name, role)
VALUES (?, ?, ?, ?, ?)
```

- ค่าทุกตัวเข้าทาง `?` ไม่ต่อข้อความ SQL (ป้องกัน SQL injection)
- ถ้าอีเมลซ้ำ MySQL ตอบ `ER_DUP_ENTRY` และโค้ดแปลงเป็น `DuplicateEmailError` (`:73-75`)
- เขียนเสร็จแล้วอ่านกลับด้วย `findById` (`:79`) เพื่อให้ได้ `created_at` จริงจาก DB

6. **ออกบัตรผ่าน:** `issue` (`auth.service.ts:74-77`) เรียก `tokens.sign({ sub: user.id, role: user.role })` แล้วคืน `{ accessToken, user }` ส่วน `sign` อยู่ที่ `token.ts:32-34` ใช้ HS256 และอายุเริ่มต้น 7 วัน (`DEFAULT_JWT_EXPIRES_IN = '7d'` ที่ `:5`)

ค. ฐานข้อมูล

ตาราง `users` (`backend/migrations/001_users.sql:12-23`)

- `id` เป็น `CHAR(36)` และเป็น PRIMARY KEY
- `email` เป็น `VARCHAR(255)` ตัวพิมพ์เล็กเสมอ และมี `UNIQUE KEY uq_users_email` (`:22`) ตัวนี้เป็นด่านสุดท้ายที่กันอีเมลซ้ำ
- `password_hash` เก็บ bcrypt hash ไม่ใช่รหัสจริง
- `display_name` ยาวไม่เกิน 100
- `role` เป็น ENUM `user`/`admin` ค่าเริ่มต้น `user`
- `created_at` และ `updated_at` (`updated_at` ขยับเองเมื่อแถวถูกแก้)

ง. ตอบกลับและหน้าจออัปเดต

1. server ตอบ 201 พร้อม JSON ที่มี `accessToken` และ `user` (id, email, displayName, role, createdAt) ใน JWT มีแค่ `sub`, `role`, `iat`, `exp` ใครก็อ่านได้แต่ปลอมไม่ได้ เพราะมีลายเซ็นจาก `JWT_SECRET`
2. **แปลงผล:** `toSession` (`api/auth.ts:13-15`) ได้ `{ token, user }` โดย `toAuthUser` (`:9-11`) หยิบแค่ `id`, `displayName`, `role` **ไม่เก็บอีเมล**
3. **เก็บการล็อกอิน:** `applySession` (`AuthProvider.tsx:75-84`)
   - `storeAuth(next)` เขียนลง `localStorage` key `fixstep.auth` (`authStorage.ts:50-56`, key ที่ `:3`)
   - `setStored(next)` อัปเดต state ใน React
   - `queryClient.clear()` ล้างแคชข้อมูลเก่า เพราะตัวตนเปลี่ยนจาก guest เป็น user
4. **ไปหน้าถัดไป:** `onSuccess` ใน `RegisterPage.tsx:27` เรียก `navigate(next ?? '/history', { replace: true })` ถ้ามี `?next=` จะกลับหน้านั้น ไม่งั้นไปหน้าประวัติ

ถ้ามีอะไรผิดพลาด

- **ข้อมูลไม่ผ่านที่หน้าจอ:** แสดงข้อความใต้ช่อง ไม่ยิง API (`RegisterPage.tsx:38`)
- **ข้อมูลไม่ผ่านที่ backend:** ได้ 400 `INVALID_AUTH_REQUEST` แล้วแสดงใน `ErrorNotice` (`:78-88`)
- **อีเมลซ้ำ:** `INSERT` ชน UNIQUE → `ER_DUP_ENTRY` → `DuplicateEmailError` (`users.store.ts:73-75`) → `emailTaken()` ได้ 409 `EMAIL_TAKEN` (`auth.service.ts:46-47`) → หน้าจอแสดงกล่อง error พร้อมลิงก์ "ไปหน้าเข้าสู่ระบบ" (`RegisterPage.tsx:80-86`)
- **server ไม่ตอบ:** `NETWORK_ERROR` (`http.ts:58-60`)

ข้อสังเกต

- ตรวจ 2 ที่ คือหน้าจอ (บอกผู้ใช้ทันที) กับ backend (ตัวตัดสินจริง) กฎหน้าจอคัดลอกมือ ถ้าจะเปลี่ยนกฎต้องแก้ทั้งสองที่
- ไม่มี rate limit กันสมัครรัว และไม่มียืนยันอีเมล
- token เก็บใน `localStorage` ถ้าเว็บมีช่องโหว่ XSS สคริปต์อ่านได้
- ข้อผิดพลาดถูกแปลงทีละชั้น: DB → store → service → HTTP

เฉลยคำถาม

- **พิมพ์รหัสผ่าน 5 ตัวแล้วกดสมัคร มีคำขอไป backend ไหม:** ไม่มี `validateRegister` เจอข้อผิดพลาดแล้ว `return` ก่อน `submit.mutate()` (`RegisterPage.tsx:38-39`) แต่ถ้ามีคนยิง API ตรงๆ backend ก็ตอบ 400 เพราะ `parsePassword` ตรวจซ้ำ
- **สมัครด้วยอีเมลที่มีอยู่แล้ว ไล่จาก DB ถึงหน้าจอ:** `INSERT` ชน UNIQUE → `ER_DUP_ENTRY` → `DuplicateEmailError` → `ApiError 409 EMAIL_TAKEN` → error handler ตอบ JSON → `request()` สร้าง `ApiError` ฝั่งหน้าจอ → `ErrorNotice` พร้อมลิงก์ไปล็อกอิน
- **ทำไมหน้าจอเก็บแค่ id, ชื่อ, role ไม่เก็บอีเมล:** `toAuthUser` (`api/auth.ts:9-11`) หยิบเท่าที่ต้องใช้แสดงผล ลดข้อมูลส่วนตัวที่ค้างใน `localStorage`
- **สมัครแล้วเป็น admin ได้ไหม:** ไม่ได้ `parseRegisterBody` เมิน `role` และ service กำหนด `Role.User` ตายตัว (`auth.service.ts:43`)

ที่ไม่ได้ทำ

- ทั้งหมดมาจากการอ่านโค้ด ไม่ได้รันเว็บหรือยิง API จริง
- ไม่ได้เปิดดู `toUserDto` (รูปร่าง JSON ที่ตอบกลับ), `emailTaken()` และเนื้อ error handler (`error-handler.ts:74-100`) ส่วนนั้นมาจากเอกสารเก่าที่เคยตรวจไว้

ช่วงที่ 2: เข้าสู่ระบบ + JWT + auth middleware

ภาพรวม: วันถัดมา สมชายมีบัญชีแล้ว เปิดเว็บแล้วกด "เข้าสู่ระบบ" หน้าจอตรวจเบื้องต้น แล้วยิง `POST /auth/login` backend หาผู้ใช้ เทียบรหัสผ่านด้วย bcrypt แล้วออก JWT หน้าจอเก็บ JWT ไว้ใน `localStorage` และแนบไปกับทุกคำขอถัดไป backend ตรวจบัตรด้วย auth middleware ก่อนให้เข้า route ที่ต้องรู้ตัวตน

ช่วงนี้มี 3 เรื่องหลัก คือ JWT คืออะไร, เก็บและแนบยังไง, backend ตรวจยังไง

JWT คืออะไร

- เป็นบัตรผ่านที่เซิร์ฟเวอร์ออกให้หลังล็อกอิน ผู้ใช้แนบมากับทุกคำขอ แทนการถามรหัสผ่านทุกครั้ง
- ข้างในบัตรมี `sub` (id ผู้ใช้), `role`, `iat` (ออกเมื่อ), `exp` (หมดเมื่อ ค่าเริ่มต้น 7 วัน) `sign` อยู่ที่ `token.ts:32-34` ใส่แค่ `sub` กับ `role`
- **ใครก็อ่านข้อมูลในบัตรได้** เพราะไม่ได้เข้ารหัส แต่ **ปลอมไม่ได้** เพราะมีลายเซ็นจาก `JWT_SECRET` (HS256) แก้ข้อมูลแม้ตัวเดียว ลายเซ็นก็ไม่ตรง
- จึงห้ามใส่ความลับลงในบัตร

ก. กดล็อกอิน (หน้าจอ → backend → DB → กลับมาเก็บบัตร)

ฝั่งหน้าจอ

1. **ไปหน้าล็อกอิน:** ลิงก์ในเมนูคือ `NavLink to="/login"` (`AppShell.tsx:71`) และ `App.tsx:48` จับคู่ `/login` กับ `<LoginPage />`
2. **หน้าเก็บค่าที่พิมพ์:** `LoginPage.tsx:21-23` มี state `email`, `password`, `fieldErrors` อ่าน `next` (หน้าที่จะกลับไปหลังล็อกอิน) ผ่าน `safeNext` (`:18`) และอ่าน `reason` (`:19`) ถ้า `reason` เป็น `expired` จะมีกล่อง "การเข้าสู่ระบบหมดอายุ" (`:50-54`)
3. **กดปุ่ม:** `<form onSubmit={handleSubmit}>` (`:61`) ปุ่ม `type="submit"` อยู่ที่ `:81-83`
4. **ตรวจเบื้องต้น:** `handleSubmit` (`:35-41`)

```
const errors = validateLogin(email, password)
setFieldErrors(errors)
if (hasErrors(errors)) return   // ไม่ผ่าน → ไม่ยิง API
submit.mutate()
```

`validateLogin` (`lib/authForm.ts:27-33`) ตรวจแค่ 2 อย่าง คืออีเมลรูปแบบถูก (`checkEmail` `:20-25`) และรหัสผ่านไม่ว่าง (`:31`) ตรวจน้อยกว่าตอนสมัครโดยตั้งใจ เพราะกฎรหัสผ่านอาจเปลี่ยนทีหลัง คนที่สมัครไว้ด้วยกฎเก่ายังต้องล็อกอินได้ (เป็นข้อสรุปจากเอกสารเก่า ไม่ได้เห็นคอมเมนต์ในโค้ดยืนยัน) 5. **สั่งส่ง:** `submit` (`:25-28`) คือ `useMutation` เรียก `login(email.trim(), password)` สำเร็จแล้ว `navigate(next ?? '/', { replace: true })` 6. **provider รับช่วง:** `login` ใน `AuthProvider.tsx:86-91` เรียก `applySession(await apiLogin(email, password))` 7. **api client:** `api/auth.ts:17-24` เรียก `request('/auth/login', { method: 'POST', body: { email, password }, anonymous: true })` `anonymous: true` = ไม่แนบ token 8. **ยิง fetch:** `request()` ใน `api/http.ts:39-86` ต่อ URL (`:41`) ข้ามการหา token เพราะ `anonymous` (`:43`) ส่งด้วย `fetch` (`:53`)

ฝั่งเซิร์ฟเวอร์

1. **ผ่านด่านแรก:** `app.ts:66` `cors` แล้ว `:67` `express.json()`
2. **เข้า router:** `app.ts:71` ส่งต่อให้ `authRoutes` แล้ว `auth.routes.ts:16-18`

```
router.post('/login', asyncRoute(async (request, response) => {
  response.status(200).json(await auth.login(parseLoginBody(request.body)));
}));
```

ไม่มี auth middleware (ยังไม่มีบัตร) ตอบ **200** (ไม่ใช่ 201 เพราะไม่ได้สร้างอะไรใหม่) 3. **ตรวจ input:** `parseLoginBody` (`auth.dto.ts:99-112`)

- อีเมล: ต้องเป็นข้อความ ตัดช่องว่างและแปลงเป็นตัวพิมพ์เล็ก (`:102`) ต้องไม่ว่างและไม่ยาวเกิน (`:103`)
- รหัสผ่าน: ต้องไม่ว่างและไม่ยาวเกิน `MAX_LOGIN_PASSWORD_LENGTH` (`:108`) ไม่มีกฎ 8–72 เหมือนตอนสมัคร
- ผิด → 400 `INVALID_AUTH_REQUEST`

4. **ตรรกะล็อกอิน:** `AuthService.login` (`auth.service.ts:55-64`)

```
const user = await this.usersStore.findByEmail(body.email);                                    //
:56
const matches = await compare(body.password, user ? user.passwordHash : this.dummyHash);       //
:58
if (!user || !matches) throw invalidCredentials();                                              //
:59-61
return this.issue(user);                                                                        //
:63
```

- `findByEmail` (`users.store.ts:55-58`) รัน

```
SELECT id, email, password_hash, display_name, role, UNIX_TIMESTAMP(created_at) AS created_epoch
  FROM users WHERE email = ? LIMIT 1
```

- `compare` เทียบรหัสที่พิมพ์กับ bcrypt hash ใน DB
- **ถ้าไม่พบอีเมล ก็ยัง `compare` กับ `dummyHash`** (สร้างไว้ที่ `:26`) เพื่อให้เวลาตอบใกล้เคียงกับกรณีพบ คนร้ายจะวัดเวลาเดาไม่ได้ว่าอีเมลไหนมีบัญชี (user enumeration) และข้อความ error ก็เหมือนกันทั้งสองกรณี
- ผิด → 401 `INVALID_CREDENTIALS` ข้อความเดียวทุกกรณี

5. **ออกบัตร:** `issue` (`auth.service.ts:74-77`) เรียก `tokens.sign({ sub: user.id, role: user.role })` แล้วคืน `{ accessToken, user }`

ฐานข้อมูล

- ตาราง `users` (`001_users.sql:12-23`) ช่วงนี้อ่านอย่างเดียว ไม่มีการเขียน

กลับมาที่หน้าจอ

1. **แปลงผล:** `toSession` (`api/auth.ts:13-15`) ได้ `{ token, user: { id, displayName, role } }` ไม่เก็บอีเมล
2. **เก็บบัตร:** `applySession` (`AuthProvider.tsx:75-84`)
   - `storeAuth(next)` เขียนลง `localStorage['fixstep.auth']` (`authStorage.ts:50-56`, key ที่ `:3`) พร้อมเก็บสำเนาในตัวแปร `memory` (`:51`)
   - `setStored(next)` อัปเดต state ใน React
   - `queryClient.clear()` ล้างแคชข้อมูลของผู้ใช้คนก่อน
   - การเขียน `localStorage` ห่อ `try/catch` ไว้ (`:52-55`) ถ้าเขียนไม่ได้ (เช่น โหมดส่วนตัวบางตัว) จะยังจำใน `memory` ของแท็บนั้นแทน ไม่ให้ล็อกอินล้ม
3. **ไปหน้าถัดไป:** `onSuccess` (`LoginPage.tsx:27`) ไป `next` หรือหน้าแรก

ข. ใช้บัตรกับทุกคำขอถัดไป + backend ตรวจ

ฝั่งหน้าจอ: แนบบัตร

1. `request()` อ่าน token จาก `getStoredToken()` (`http.ts:43`, ฟังก์ชันอยู่ที่ `authStorage.ts:46-48`) อ่านจากที่เก็บตรงๆ ไม่ผ่าน React state จึงไม่มีช่วงที่คำขอยังใช้บัตรเก่าหลังล็อกอินหรือออกจากระบบ
2. ใส่หัว `Authorization: Bearer <token>` (`http.ts:48`) ยกเว้นคำขอ `anonymous`

ฝั่ง backend: ตรวจบัตร (`auth.middleware.ts`)

`readUser` (`:20-42`) ทำตามลำดับ

1. ไม่มีหัว `Authorization` → คืน `undefined` (`:22-24`) ให้ตัวเรียกตัดสินว่าเป็น guest หรือ 401
2. มีหัวแต่รูปแบบผิด (ต้องเป็น `Bearer <token>` สองชิ้นพอดี) → 401 (`:26-29`)
3. ตรวจลายเซ็นและวันหมดอายุด้วย `tokens.verify` (`:33`, ตัวฟังก์ชันที่ `token.ts:37-40` บังคับ `algorithms: ['HS256']`) ล้ม → 401 (`:34-36`)
4. ตรวจรูปข้อมูลในบัตร: `sub` ต้องเป็นข้อความไม่ว่าง และ `role` ต้องเป็นบทบาทจริง (`isRole`) ไม่งั้น 401 (`:38-40`) เพราะลายเซ็นถูกไม่ได้แปลว่าข้อมูลถูกรูปแบบ
5. ผ่าน → คืน `{ sub, role }` (`:41`) ข้อความ 401 เหมือนกันทุกสาเหตุ (`:17-18`) ไม่บอกว่าผิดเพราะอะไร

middleware 3 ตัวที่ใช้ `readUser` (`createAuthMiddleware` `:44`)

- **`optionalAuth`** (`:49-57`): ใช้เมื่อ guest หรือผู้ล็อกอินก็ได้ ไม่มีบัตรผ่านเป็น guest แต่ **มีบัตรที่เสียได้ 401** ไม่แอบลดเป็น guest เพราะประวัติจะหายเงียบๆ
- **`requireAuth`** (`:59-68`): ต้องล็อกอิน ไม่มีบัตร 401 บัตรเสีย 401
- **`requireAdmin`** (`:70-80`): ต้องวางหลัง `requireAuth` ไม่มี `request.user` → 401 และไม่ใช่ admin → **403** `FORBIDDEN` (`:75-76`)

ผลลัพธ์วางไว้ที่ `request.user` และ handler อ่านต่อด้วย `currentUser(request)` (`:13-15`)

ตัวอย่างที่ติดจริง: `/auth/me` ใช้ `requireAuth` (`auth.routes.ts:21`), `/traversal/*` กับ `/history/*` ใช้ `optionalAuth` (`app.ts:73-74`), `/admin/*` ใช้ `requireAuth` ตามด้วย `requireAdmin` ทีละ route (`admin.routes.ts:26-27`)

ตัวอย่าง: ตอนเปิดเว็บยืนยันตัวตน

1. `AuthProvider.tsx:56-73` ตอนโหลดแอป ถ้ามี token ที่จำไว้ จะเรียก `getMe()` (`api/auth.ts:35-37`) → `GET /auth/me` (`auth.routes.ts:21-23`) → `AuthService.me` (`auth.service.ts:66-72`) → `findById` (`users.store.ts:60-63`)
2. ถ้าสำเร็จ จะเอา role และชื่อจากเซิร์ฟเวอร์ทับค่าที่จำไว้ (`:63-66`) ระหว่างรอ สถานะเป็น `verifying` (`:109`)
3. ถ้าล้มเหลว (`:67-69`) แค่เลิกสถานะ `verifying` ไม่ล้าง token จึงใช้ค่าที่จำไว้ต่อ

ค. บัตรเสียหรือหมดอายุ และออกจากระบบ

1. **backend ตอบ 401:** `unauthenticated()` (`auth.middleware.ts:18`) → ส่งเข้า error handler กลาง
2. **หน้าจอรู้ว่า token เสีย:** `http.ts:70-72` เงื่อนไขคือ `status === 401 && token !== undefined` เฉพาะคำขอที่แนบ token เท่านั้น แล้วเรียก `unauthorizedHandler(token)`
3. **ล้างแล้วพาไปล็อกอินใหม่:** handler ที่ `AuthProvider.tsx:41-51`
   - ถ้า token ที่ใช้ไม่ใช่ token ปัจจุบัน (คำขอเก่าตอบช้าหลังล็อกอินใหม่) ให้เมิน (`:43`)
   - `clearStoredAuth`, `setStored(null)`, `queryClient.clear()` (`:45-48`)
   - `navigate(loginUrl('expired', หน้าเดิม))` (`:50`) ไป `/login?reason=expired&next=...`
4. **ออกจากระบบ:** ปุ่มที่ `AppShell.tsx:84` เรียก `logout` (`AuthProvider.tsx:100-106`) ล้าง localStorage กับแคชแล้วกลับหน้าแรก **ฝั่งเซิร์ฟเวอร์ไม่มีอะไรเกิดขึ้น**

ถ้ามีอะไรผิดพลาด

- **อีเมล/รหัสผ่านไม่ผ่านที่หน้าจอ:** แสดงใต้ช่อง ไม่ยิง API
- **ข้อมูลไม่ครบหรือรูปแบบผิดที่ backend:** 400 `INVALID_AUTH_REQUEST`
- **อีเมลหรือรหัสผ่านไม่ถูก:** 401 `INVALID_CREDENTIALS` แสดงกล่อง error ที่หน้า (`LoginPage.tsx:79`) ไม่เด้งไปหน้าล็อกอินซ้ำ เพราะ `anonymous: true` ทำให้ `token` เป็น `undefined`
- **บัตรหมดอายุหรือเสียระหว่างใช้งาน:** 401 `UNAUTHENTICATED` → ล้างล็อกอิน → `/login?reason=expired&next=...`
- **ผู้ใช้ธรรมดาเรียก `/admin/*`:** 403 `FORBIDDEN`
- **เซิร์ฟเวอร์ไม่ตอบ:** `NETWORK_ERROR` (`http.ts:58-60`)

ข้อสังเกต

- **ไม่มี logout ฝั่งเซิร์ฟเวอร์:** token ใช้ได้จนหมดอายุ (7 วัน) แม้กดออกจากระบบ
- token อยู่ใน `localStorage` ถ้าเว็บมีช่องโหว่ XSS สคริปต์อ่านได้
- `role` ในบัตรเชื่อตามที่ออก ไม่ query DB ทุกคำขอ ถ้าเปลี่ยน role ของผู้ใช้ ต้องรอบัตรเก่าหมดอายุ (`/auth/me` อ่าน DB เฉพาะตอนเปิดเว็บ และเปลี่ยนแค่ข้อมูลที่หน้าจอจำ ไม่ได้ออกบัตรใหม่)
- `AdminRoute` ฝั่งหน้าจอเป็น UX เท่านั้น ตัวบังคับจริงคือ `requireAdmin`

- ไม่มี rate limit กันเดารหัสผ่าน

เฉลยคำถาม

- **แก้ `role` ใน localStorage เป็น `admin` แล้วเปิด `/admin/analytics` จะเห็นอะไร:** หน้าจออาจแสดงหน้าได้ชั่วขณะ แต่ตอนเปิดเว็บ `GET /auth/me` เอา role จริงจากเซิร์ฟเวอร์ทับ (`AuthProvider.tsx:63-66`) จึงกลับเป็น `user` และต่อให้ข้ามได้ ทุกคำขอ `/admin/*` ใช้ token ที่ `role` ถูกเซ็นไว้ใน JWT แก้ไม่ได้ จึงโดน `requireAdmin` ตอบ 403 และหน้าจอไม่ได้ข้อมูลเลย
- **กดออกจากระบบแล้ว token เดิมยังใช้ได้ไหม:** ใช้ได้จนหมดอายุ ถ้ามีใครได้ token ไปก่อน เพราะ backend ไม่มี logout endpoint และไม่เก็บ token ใน DB ปุ่มออกแค่ลบ token ออกจากเบราว์เซอร์
- **ล็อกอินผิดได้ 401 ทำไมหน้าจอไม่เด้งไปหน้าล็อกอินซ้ำ:** เพราะเงื่อนไขที่ `http.ts:70` ต้องมี token แนบไปด้วย คำขอล็อกอินเป็น `anonymous` จึงไม่มี token และ 401 ในกรณีนี้หมายถึงรหัสผิด ไม่ใช่ token เสีย
- **ทำไมไม่พบอีเมลก็ยัง `compare` กับ `dummyHash`:** กันการวัดเวลาตอบ ถ้าข้าม `compare` เมื่อไม่พบอีเมล จะตอบเร็วกว่ากรณีพบ คนร้ายเดาได้ว่าอีเมลไหนมีบัญชี
- **ส่ง `Authorization: Bearer` (ไม่มี token ตามหลัง) ไปที่ `/auth/me` และ `/history`:** 401 ทั้งสองที่ เพราะมีหัวแต่รูปแบบผิด (`auth.middleware.ts:26-29`) `optionalAuth` ก็ไม่ปล่อยเป็น guest เพราะมีหัวแล้วใช้ไม่ได้ = 401
- **ถ้าลำดับเป็น `[requireAdmin, requireAuth]`:** `requireAdmin` รันก่อนจึงยังไม่มี `request.user` ทุกคนได้ 401 รวมถึง admin จริง (`:71-74`)

ที่ไม่ได้ทำ

- ทั้งหมดมาจากการอ่านโค้ด ไม่ได้รันเว็บหรือยิง API จริง
- ไม่ได้เปิดดูตัวฟังก์ชัน `invalidCredentials()` กับ `emailTaken()` ว่าอยู่ไฟล์ไหน ข้อความ "401 `INVALID_CREDENTIALS`" มาจากเอกสารเก่า
- ไม่ได้ตรวจ `authRedirect.ts` (`safeNext`, `loginUrl`) ว่าทำอะไรบ้างเกินชื่อฟังก์ชัน

ช่วงที่ 3: เลือกเครื่อง → เลือกรุ่น

เล่าต่อ: สมชายล็อกอินแล้ว กด "เลือกเครื่องที่มีปัญหา" จากหน้าแรก (`HomePage.tsx:17`) ช่วงนี้ **ไม่มีการยิง SQL ตอนใช้งานเลย** เพราะข้อมูลทั้งหมดอยู่ในหน่วยความจำของเซิร์ฟเวอร์ตั้งแต่ตอนบูต นี่คือหัวใจของช่วงนี้

ตอนที่ 1: หน้าจอ (Frontend)

หน้า `pages/DevicesPage.tsx` (route `/devices` — `App.tsx:42`) มี 2 ขั้นในหน้าเดียว สลับกันด้วย `?category=` ใน URL

1. **เปิดหน้า:** `useManuals()` (`DevicesPage.tsx:29`, `api/queries.ts:70-72`) → `listManuals()` (`api/traversal.ts:24-26`) → **`GET /traversal/manuals`** ไม่มี body ไม่ต้องล็อกอิน `staleTime: Infinity` = โหลดครั้งเดียวไม่หมดอายุ เพราะรายการเปลี่ยนเฉพาะตอนมีคู่มือใหม่
2. **ขั้น 1 เลือกประเภทเครื่อง:** `CategoryStep` (`:71-93`) ดูว่าในรายการรุ่นมีประเภทอะไรบ้าง แสดงเป็นการ์ดพร้อม "N รุ่นในระบบ" กดการ์ด (`:81`) → `setParams({ category })` URL เปลี่ยนเป็น `/devices?category=...`
3. **ขั้น 2 เลือกรุ่น:** `ModelStep` (`:95-157`) แสดงรุ่นของประเภทนั้น
   - **ช่องพิมพ์เลขรุ่น** (`:119`) กรองรายการ **ที่โหลดมาแล้ว** ฝั่งหน้าจอ ด้วย `manualMatches` (`lib/models.ts:22-24`) ที่ทำให้ตัวอักษรเป็นตัวใหญ่และตัดช่องว่าง/ขีด แล้วเทียบกับ pattern ของรุ่น (รองรับ `*` แทนตัวอักษรใดก็ได้, pattern หลายตัวคั่นด้วย `/`, และชื่อเรียกอื่น `modelAliases`)
   - **ปุ่ม "ฉันไม่ทราบรุ่น"** (`:151-153`) สลับ `isUnknown`: ซ่อนช่องกรอง แสดงคำเตือนว่าขั้นตอนอาจไม่ตรงกับเครื่อง (`:123-127`) แล้วให้เลือกรุ่นที่ใกล้เคียงเอง **ระบบไม่เลือกให้**
   - **กดเลือกรุ่น** (`pick` `:104-106`) → `navigate(symptomsPath(manualId, isUnknown ? 'unknown' : 'selected'))` ไปที่ `/symptoms?manual=<รหัสรุ่น>[&unknown=1]` (`lib/modelSelection.ts:17-21`)
4. **รุ่นที่เลือกเก็บไว้ที่ไหน:** ใน **URL** (`?manual=...`) อ่านกลับด้วย `useChosenModel()` (`lib/modelSelection.ts:9-14`) ไม่มี store กดย้อนกลับ/รีเฟรช/แชร์ลิงก์แล้วยังอยู่รุ่นเดิม

ไม่มีช่องกรอก device/brand/model แบบอิสระ: ประเภทเครื่องเป็นรายการคงที่ ยี่ห้อมากับข้อมูลคู่มือ มีเฉพาะช่อง "เลขรุ่นเพื่อกรอง" ที่กรองรายการเดิม

ตอนที่ 2: เซิร์ฟเวอร์ (Backend)

คำขอ → `cors` → `express.json` → `/traversal` → `traversal.routes.ts`

`traversal.routes.ts:32-34` `GET /manuals` (ไม่มี auth middleware) → `TraversalService.listManuals` (`traversal.service.ts:48-50`) → `GraphRepository.listManuals` (`graph.repository.ts:99-118`)

`listManuals` **ไม่มีตารางคู่มือในหน่วยความจำ** จึงสร้างจากผัง: วนทุกผัง ถ้าคู่มือนั้นเคยเจอแล้วบวก `graphCount` ไม่เคยเจอสร้างรายการใหม่ แนบ `displayName` / `modelAliases` จาก `manualExtras` ได้ผลเป็นรายการรุ่นพร้อมจำนวนอาการ

แล้วข้อมูลในหน่วยความจำมาจากไหน → ตอนบูต

`app.ts:58` `await graphs.load()` (ต้องเสร็จก่อนรับคำขอ) → `GraphRepository.loadAll` (`graph.repository.ts:120-172`) ยิง SQL สองคำสั่ง:

```
-- :121-129  ผังทั้งหมด + ข้อมูลคู่มือ
SELECT g.graph_id, g.manual_id, g.entry_symptom, g.entry_symptom_th, g.entry_symptom_aliases,
       g.source, g.page_start, g.page_end, g.source_chunk_id, g.entry_node,
       m.device_category, m.brand, m.model_pattern, m.display_name, m.model_aliases
  FROM graphs g JOIN manuals m ON m.manual_id = g.manual_id ORDER BY g.graph_id
-- :133-138  โหนดทั้งหมด
SELECT graph_id, node_id, node_type, text_content, on_yes, on_no, next_node, on_invalid,
       input_type, input_pattern, store_as, outcome_kind, safety_critical, safety_warning
  FROM nodes ORDER BY graph_id, display_order
```

แล้วประกอบเป็น `Map` ในหน่วยความจำ (สร้างใหม่ก่อนค่อยสลับ ถ้าข้อมูลเสีย **บูตไม่ขึ้น** ตามที่ออกแบบ) · อุปกรณ์โหลดแบบเดียวกันที่ `app.ts:59` (`equipment.repository.ts:30-37`)

ทำไมเก็บในหน่วยความจำ: ผัง ~600 ผังเปลี่ยนน้อย ทุกคำขอใช้ร่วมกัน อ่านจากหน่วยความจำเร็วกว่ายิง DB ซ้ำและ engine เดินผังได้เร็ว **ราคาที่จ่าย:** ข้อมูลใหม่ที่ seed ด้วยสคริปต์ต้องรีสตาร์ต backend ถึงเห็น ส่วนผังที่ admin เผยแพร่ระบบโหลดซ้ำให้เอง (`drafts.service.ts:199`)

ทางเลือก: เลือกจากรายการ "อาการทั้งหมด"

`pages/AllSymptomsPage.tsx` → `useGraphs(manualId)` (`queries.ts:61-67`) → **`GET /traversal/graphs?manualId=`** (`api/traversal.ts:19-22`) → `traversal.routes.ts:27-29` (`parseOptionalManualId` ตรวจ) → `TraversalService.listGraphs` (`:42-45` กรองด้วย `manualId`) → `GraphRepository.listAll` (`graph.repository.ts:87-97`) ได้รายการอาการแบบย่อ ไม่ต้องพึ่งระบบค้นหา (ใช้ได้เสมอแม้ระบบค้นหาปิด)

ตอนที่ 3: ฐานข้อมูล

อ่านตอนบูตเท่านั้น: `graphs`, `manuals`, `nodes` (migration `002_graphs.sql`) และ `equipment`, `equipment_usage` (`003_equipment.sql`) · **ตอนผู้ใช้เลือกเครื่อง ไม่แตะ DB**

ตอนที่ 4: คำตอบกลับ

```
// GET /traversal/manuals → 200
[ { "manualId": "samsung_...", "deviceCategory": "microwave", "brand": "Samsung",
    "modelPattern": "GE83MDT...", "displayName": "...", "modelAliases": ["..."], "graphCount": 21
} ]
```

หน้าจอแสดง `brand + ชื่อรุ่น` กับ `{graphCount} อาการ` (`DevicesPage.tsx:139-141`) ชื่อรุ่นใช้ `displayName` ถ้ามี ไม่มีใช้ `modelPattern` (ไม่เดาชื่อ)

ข้อสังเกต

- `/traversal/manuals` และ `/graphs` ไม่ตรวจ token เลย จึงไม่พังเพราะ token เสีย (`traversal.routes.ts:8`)
- รายการรุ่นถูก invalidate ตอน admin เผยแพร่ (`queries.ts:318-321`) จึงตั้ง `staleTime: Infinity` ได้
- ตอนไม่ทราบรุ่น ระบบ **บันทึก** ว่า `unknown` ไว้แยกเคสตอนวัดผล ไม่เปลี่ยนขั้นตอน (ช่วงถัดไปจะเห็นตอนส่งเข้า session)

คำถามเช็ก + เฉลย

1. ตอนสมชายเปิด `/devices` มีการยิง SQL ไหม? เฉลย: ไม่มี `listManuals` อ่านจาก `Map` ในหน่วยความจำ SQL ที่เกี่ยวข้องยิงครั้งเดียวตอนบูต (`graph.repository.ts:121-139`)

2. สมชายกด "ฉันไม่ทราบรุ่น" แล้วเลือกรุ่นหนึ่ง ระบบเก็บอะไร ใช้ทำอะไร? เฉลย: URL ได้ `unknown=1` (`modelSelection.ts:17-21`) พอเริ่มตรวจ หน้าจอส่ง `modelSelection:'unknown'` ไป backend → เก็บในคอลัมน์ `sessions.model_selection` (`session.store.ts:42-51`) ใช้แยกเคสตอนวัดผลเท่านั้น **ไม่เปลี่ยนขั้นตอน** และหน้าตรวจแสดงกล่องเตือนว่าขั้นตอนอาจไม่ตรงกับเครื่อง (`SessionPage.tsx:155-157`)

3. ถ้า admin เพิ่งเผยแพร่คู่มือใหม่ ผู้ใช้ที่เปิดหน้า `/devices` ค้างไว้จะเห็นรุ่นใหม่ไหม? เฉลย: ฝั่ง backend ตอบรุ่นใหม่ทันที (publish แล้ว `graphRepository.reload()` `drafts.service.ts:199`) ฝั่งเบราว์เซอร์ของ admin เองล้างแคช `['manuals']` (`queries.ts:318-321`) แต่ผู้ใช้คนอื่นที่เปิดหน้าค้างไว้ แคชไม่หมดอายุ (`staleTime: Infinity`) จึงเห็นรุ่นใหม่เมื่อรีเฟรชหน้า (อ่านจากโค้ด ไม่ได้ทดสอบ)

ไปต่อ **ช่วงที่ 4: พิมพ์อาการ → ค้นหาด้วย embedding → เริ่มตรวจ** (ช่วงที่ยาวและน่าสนใจที่สุด) ได้เลยครับ

ช่วงที่ 4: พิมพ์อาการ → ค้นหาด้วย embedding → เลือก → เริ่มตรวจ

เล่าต่อ: สมชายเลือกรุ่นไมโครเวฟแล้ว หน้า `/symptoms?manual=...` เปิดขึ้น เขาพิมพ์ "ไมโครเวฟไม่ร้อน" ช่วงนี้คือ **"RAG" ตามชื่อเรียกในโจทย์** แต่ของจริงไม่มีขั้นที่ AI เขียนคำตอบ ระบบทำแค่ **ค้นหาอาการที่ใกล้เคียง** แล้ว **ให้ผู้ใช้เลือกเอง** จากนั้นค่อยเดินผังที่เขียนไว้ตามคู่มือ

ตอนที่ 1: หน้าจอค้นหา (Frontend)

หน้า `pages/SymptomsPage.tsx`

1. ถ้ายังไม่มีรุ่นใน URL → เด้งกลับ `/devices` (`:52` ไม่เดารุ่นให้)
2. กรอกข้อความที่ `components/SymptomSearchForm.tsx` (ช่องพิมพ์ `:29` จำกัด 200 ตัว, ปุ่ม "ค้นหา" `:30-32` กดไม่ได้จนกว่า `checkQuery` จะ `ok` `lib/textInput.ts:31-36`)
3. กดค้นหา → `handleSearch` (`SymptomsPage.tsx:43-46`) → `search.mutate({ query, manualId })` ผ่าน `useSymptomSearch` (`api/queries.ts:97`)
   - ใช้ `useMutation` ไม่ใช่ `useQuery` และไม่แคช เพราะผลขึ้นกับสถานะเซิร์ฟเวอร์ (เช่นตอนแรกยังโหลดโมเดลอยู่) ไม่ควรจำผลเก่า
4. `api/traversal.ts:47-52` → **`POST /symptom-search`** body `{ "query": "ไมโครเวฟไม่ร้อน", "manualId": "samsung_..." }`
5. ผลที่หน้าจอ: มีผล → `SymptomSearchResults.tsx:21-32` แสดงแถวละอาการพร้อม **คะแนนความคล้าย** (`SymptomRow.tsx:28-33`) · ไม่มีผล → ข้อความ "ไม่พบอาการที่ใกล้เคียง... ระบบไม่เดาอาการให้" (`:13-19`) · error บางชนิด (`INVALID_QUERY`, `SEARCH_NOT_READY`, `SEARCH_UNAVAILABLE`) แสดงเป็นกล่องสีฟ้าแทนสีแดง เพราะไม่ใช่ระบบพัง ผู้ใช้ยังไปหน้า "อาการทั้งหมด" ได้ (`SymptomsPage.tsx:31`, `:102-111`)

ตอนที่ 2: เซิร์ฟเวอร์ค้นหา (Backend)

`symptom-search.routes.ts:9-13` (ไม่มี auth middleware ใครก็ค้นได้) → `parseSearchBody` (`symptom-search.dto.ts:14`) → `SymptomSearchService.search` (`symptom-search.service.ts:160-192`)

ลำดับใน `search`:

1. ระบบยังโหลดโมเดล → **503** **`SEARCH_NOT_READY`** (`:161`) · ปิด/ล้ม → 503 `SEARCH_UNAVAILABLE` (`:162`)
2. **แปลงคำค้นเป็นเวกเตอร์** `embedding.embed(query)` (`:164`)
3. **กรองเฉพาะผังของรุ่นที่เลือก** ในหน่วยความจำด้วย `manualId` (`:165-168`) ไม่ใช่ SQL
4. **จัดอันดับ** `rankGraphs` (`symptom-matcher.ts:64-88`)
5. **ตัดผล** `selectMatches(ranked, 0.55, 3)` (`:172`)

สองแนวคิดที่ต้องเข้าใจ

Embedding (เวกเตอร์): โมเดล AI แปลง "ข้อความ" เป็น "ชุดตัวเลข 384 ตัว" ที่ข้อความ **ความหมายใกล้กัน** ได้ตัวเลขใกล้กัน เช่น "ไมโครเวฟไม่ร้อน" กับ "อุ่นอาหารไม่ขึ้น" ได้เวกเตอร์ที่ชี้ไปทางเดียวกัน แม้คำไม่ตรงกัน นี่คือเหตุผลที่ค้น

ด้วยความหมายได้ ไม่ใช่แค่ตรงคำ (`embedding.service.ts:46-55`: mean pooling + normalize, โมเดล `Xenova/paraphrase-multilingual-MiniLM-L12-v2` `:5-6`)

Cosine similarity: วัดว่า "เวกเตอร์สองตัวชี้ไปทางเดียวกันแค่ไหน" (`symptom-matcher.ts:20-41`) ได้ค่า 1 = ทิศเดียวกันพอดี, ยิ่งต่ำยิ่งต่างความหมาย ระบบเทียบคำค้นกับ **ทุกข้อความของทุกผัง** แล้วเก็บ **คะแนนสูงสุดต่อผัง** (`:64-88`) เรียงมากไปน้อย

ตัดผล: เอาเฉพาะคะแนน **≥ 0.55** และไม่เกิน **3 ผัง** (`MATCH_THRESHOLD` `:11`, `MAX_RESULTS` `:12`) เทียบกับค่าเต็มก่อนค่อยปัดเศษ 3 ตำแหน่งตอนส่งออก (`round3` `:180`) ถ้าไม่มีผังไหนถึงเกณฑ์ → `matches: []` **เหตุผลที่เลือก 0.55 และ 3: ไม่ทราบ** ไม่ได้ระบุในโค้ด

ตอนที่ 3: ดัชนีเวกเตอร์ถูกสร้างเมื่อไร (ตอนบูต)

ผังไม่ได้เก็บเวกเตอร์ไว้ใน MySQL แต่สร้างใหม่ในหน่วยความจำทุกครั้งที่เซิร์ฟเวอร์บูต

1. `app.ts:60` `symptomSearch.start()` **ไม่รอ** (โมเดลโหลดช้า ถ้ารอ เซิร์ฟเวอร์เปิดรับคำขอไม่ได้) ต้องตั้ง env `SYMPTOM_SEARCH_ENABLED=true` (`symptom-search.service.ts:45-54`)
2. `prepareIndex` (`:56-102`): โหลดโมเดล (`embedding.service.ts:23-34` จากโฟลเดอร์ `backend/model` `allowRemoteModels=false`) → `buildSymptomTexts` (`symptom-matcher.ts:43-62`) ดึงข้อความของทุกผัง = ชื่อไทย + ชื่ออังกฤษ + คำเรียกอื่น (aliases) → แปลงเป็นเวกเตอร์ทีละข้อความ (ข้อความซ้ำใช้เวกเตอร์ร่วมกัน `:78-88`) → เก็บใน `this.index`
3. สถานะมี 4 แบบ `disabled → loading → ready / failed` ดูได้ที่ `GET /symptom-search/status` (`routes :15-17`)
4. **ผังใหม่หลัง admin เผยแพร่:** `addGraphs` (`:112-158`) แปลงเฉพาะผังใหม่เข้าดัชนีที่ทำงานอยู่ ไม่ทำใหม่หมด

ตอนที่ 4: ผู้ใช้เลือกผล → เริ่มตรวจ

1. กดแถว (`SymptomRow.tsx:23`) → `SymptomsPage.tsx:83` → `start(graphId, { query, manualId, modelSelection })` (`lib/useStartSymptom.ts:27-33`)
2. `useStartSession` (`queries.ts:118`) → `api/traversal.ts:37-44` → **`POST /traversal/sessions`** body `{ graphId, query, manualId, modelSelection }` + `headers: ownerHeaders()` (ล็อกอิน: token / guest: `X-Client-Id` ดูเรื่อง identity ที่เพิ่งเรียน)
3. **Backend:** `traversal.routes.ts:40-48` ด่าน `optionalAuth` → `parseStartSessionBody` (`traversal.dto.ts:92-117`) → `resolveOptionalOwner` (`session-owner.ts:14-23`) → `TraversalService.startSession` (`traversal.service.ts:56-73`):
   - `requireGraph` ไม่พบ → 404 · `manualId` ไม่ตรงผัง → 400 `GRAPH_MANUAL_MISMATCH` (`:64-66`)
   - `computeConfidence` (`:186-193`) → `symptomSearch.scoreGraph(query, graphId)` (`symptom-search.service.ts:193-199`) หาคะแนนของผังที่เลือก
   - `engineStartSession` (`traversal-engine.ts:4-24`) สร้าง `sessionId` (UUID) ตั้งโหนดแรก
   - `sessionStore.create` (`session.store.ts:38-57`):

```
INSERT INTO sessions
  (session_id, graph_id, current_node_id, status, variables, confidence, model_selection,
client_id, user_id, expires_at)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, NOW() + INTERVAL ? HOUR)   -- 24 ชั่วโมง
```

4. ตอบ **201** (รูปเดียวกับ session ในช่วง 5) → หน้าจอใส่ผลลงแคชทันที (`useCacheSession` `queries.ts:103`) แล้ว `navigate('/session/:id')` (`useStartSymptom.ts:31`) หน้าตรวจเปิดมาพร้อมข้อมูลโดยไม่ยิงซ้ำ

ที่ไม่ได้เกิดในช่วงนี้ (สำคัญ)

- **ไม่มีการเรียก LLM** ตอนค้นหาและตอนเริ่ม session (ค้นแล้วใน `symptom-search/`, `traversal/`) ไม่มี prompt สร้างคำตอบ LLM ถูกเรียกในช่วง 5 (เรียบเรียงถ้อยคำ) และฝั่ง admin เท่านั้น
- **ไม่มีการเก็บเวกเตอร์ใน DB** และ **ไม่ใช่ SQL filter** กรองด้วย `manualId` ในหน่วยความจำ

ข้อสังเกต

- "confidence" คือ **คะแนนความคล้ายของข้อความ ไม่ใช่ความน่าจะเป็นที่ถูก** หน้าจอบอกชัดและไม่แปลงเป็น % (`textInput.ts:38-47`)
- `scoreGraph` แปลงคำค้นเป็นเวกเตอร์ **ซ้ำอีกครั้ง** ตอนเริ่ม session (ค้นหาทำไปแล้วหนึ่งรอบ) ทำงานซ้ำซ้อนเล็กน้อย
- ข้อความ "ติดต่อศูนย์บริการ **Samsung**" ตายตัวในหน้าจอ (`SymptomSearchResults.tsx:16`)
- ถ้า `SYMPTOM_SEARCH_ENABLED` ไม่ใช่ `true` ค้นหาตอบ 503 ผู้ใช้ยังเลือกจาก "อาการทั้งหมด" ได้

คำถามเช็ก + เฉลย

1. ทำไมตอนเริ่ม session ต้องส่ง `query` ไปด้วย? เฉลย: เพื่อให้ backend คำนวณ "คะแนนความคล้าย" ของผังที่เลือก เก็บใน `sessions.confidence` ไว้แสดงผลและวัดผล **ไม่มีผลต่อขั้นตอนที่เดิน** ถ้าเริ่มจากรายการ "อาการทั้งหมด" ไม่ส่ง `query` ค่าจึงเป็น `NULL` (`useStartSymptom.ts:23-26`, `traversal.service.ts:186-187`)

2. สมชายเลือกรุ่นไมโครเวฟ แต่พิมพ์ว่า "ตู้เย็นไม่เย็น" จะเกิดอะไร? เฉลย: ตามตรรกะโค้ด ระบบกรองเฉพาะผังของรุ่นที่เลือก (`symptom-search.service.ts:165-168`) จึงเทียบกับอาการของไมโครเวฟเท่านั้น คะแนนน่าจะต่ำกว่า 0.55 → `matches: []` → หน้าจอแสดง "ไม่พบอาการที่ใกล้เคียง" พร้อมทางไป "อาการทั้งหมด" (ไม่ได้ทดสอบจริง)

3. backend เพิ่งบูต สมชายกดค้นหาทันที จะเห็นอะไร? เฉลย: โมเดลยังโหลด (`state='loading'`) → **503** **`SEARCH_NOT_READY`** (`symptom-search.service.ts:161`) หน้าจอแสดงกล่องสีฟ้า ไม่ใช่ error แดง (`SymptomsPage.tsx:31`, `:102-111`) และชวนใช้ "อาการทั้งหมด" ระบบไม่ล่ม เพราะ `start()` ไม่รอโมเดล

4. ค่า 0.55 และ top 3 เลือกด้วยเหตุผลอะไร? เฉลย: **ไม่ทราบ** ในโค้ดไม่ได้ระบุเหตุผล (และเอกสารบันทึกว่าเกณฑ์ยังไม่ได้ประเมินด้วยชุดทดสอบที่มนุษย์ตรวจ) ถ้ากรรมการถาม ตอบตรงๆ ว่าไม่ทราบเหตุผลที่เลือก และบอกว่าเป็นเกณฑ์เสนอรายการให้เลือก ไม่ใช่ตัวตัดสินความถูกต้อง

ไปต่อ **ช่วงที่ 5: เดินผังทีละขั้น** (ตอบ ใช่/ไม่ใช่ ด่านความปลอดภัย และที่มาของแต่ละขั้น) ได้เลยครับ

ช่วงที่ 5: เดินผังทีละขั้น

เล่าต่อ: สมชายเลือกอาการ "ไมโครเวฟไม่ร้อน" แล้ว ระบบพาไปหน้า `/session/<id>` ช่วงนี้คือหัวใจของระบบ: **ขั้นถัดไปถูกตัดสินโดยโค้ด (เครื่องสถานะ) ไม่ผ่าน AI** หน้าจอแค่แสดงขั้นที่เซิร์ฟเวอร์ส่งมาและส่งสิ่งที่ผู้ใช้ทำกลับไป

> ⚠ ไฟล์ `traversal.routes.ts` (94→52 บรรทัด) และ `traversal.service.ts` (193→191) ถูกลบคอมเมนต์ไปแล้ว เลขบรรทัดที่ผมเคยให้ในช่วง 3–4 ของทั้งสองไฟล์จึงเลื่อน ช่วงนี้ใช้เลขปัจจุบันที่ตรวจแล้ว (ท้ายข้อความมีข้อเสนอรีเฟรชเอกสาร)

ตอนที่ 1: หน้าจอได้ข้อมูลมาจากไหน

`SessionPage.tsx` (route `/session/:sessionId` — `App.tsx:47`)

- **ปกติไม่ต้องยิงเซิร์ฟเวอร์เลย:** ตอนเริ่ม session คำตอบถูกใส่ในแคชไว้แล้ว (`useCacheSession` `api/queries.ts:103`) `useSession` (`:53`) อ่านจากแคช
- **เปิดลิงก์ตรง/กด F5** แคชว่าง → `GET /traversal/sessions/:id` (`api/traversal.ts:54-56`) → `traversal.routes.ts:27-30` (`optionalAuth`) → `TraversalService.getSession` (`traversal.service.ts:74`): `authorize` `:136` → `requireSession` `:148` → `SessionStore.find` (อ่าน `sessions` + `session_history`) → `getCurrentNode` ของ engine

ตอนที่ 2: หน้าจอแสดงอะไร (ตรงกับหัวข้อ "คำตอบ" ในโจทย์)

`SessionPage.tsx:149-176` ประกอบหลาย component:

ส่วน	component:บรรทัด	ข้อมูลมาจาก
Problem (ชื่ออาการ + ยี่ห้อ/รุ่น)	`SessionHeader.tsx:19`, `:22-27`	รายการอาการที่โหลดไว้ (`SessionPage.tsx:42`)
Confidence	`SessionHeader.tsx:34-38` "คะแนนความคล้าย 0.812 (ข้อมูลประกอบ)" ซ่อนถ้า `null`	`session.confidence`
Solution (ขั้นตอนปัจจุบัน)	`step/StepView.tsx:24-44` เลือกหน้าตาตามชนิดโหนด	`session.node`
Safety	`step/SafetyGateStep.tsx:8-36` กล่องแดง + ช่องติ๊ก + ปุ่ม	`node.safetyWarning`, `requiresSafetyConfirmation`
References	`step/ReferenceLine.tsx:23-36` "ที่มาของขั้นตอนนี้: คู่มือ … หน้า 12–14" ติดทุกขั้นผ่าน `StepCard.tsx:37`	`node.reference`
อุปกรณ์ที่อาจต้องใช้	`EquipmentPanel.tsx:31-70`	`session.equipment`

ส่วน	component:บรรทัด	ข้อมูลมาจาก
ข้อความที่ AI ปรับถ้อยคำ	`RephraseNote.tsx` ปุ่มดูต้นฉบับ (`SessionPage.tsx:173`)	`session.displayText`
เส้นทางที่ผ่านมา	`step/PathRail.tsx:41-66` + `lib/trail.ts:12`	state `trail` ในหน้า (`SessionPage.tsx:45`)

Possible Cause: ไม่พบ (ผังไม่มีฟิลด์นี้ ตั้งใจไม่ใส่)

`StepView` เลือกหน้าตาอย่างไร (`StepView.tsx:24-44`)

```
node.isTerminal              → OutcomeStep      (ข้อสรุป + ฟอร์มบันทึกผล)
node.requiresSafetyConfirmation → SafetyGateStep (ติ๊กยืนยันก่อนกดไปต่อ)
node.type === 'checkpoint'   → CheckpointStep   (ปุ่ม ใช่ / ไม่ใช่)       :47-60
node.type === 'input'        → InputStep        (ช่องกรอก เช่น รหัสแจ้งเตือน)
อื่นๆ (instruction)          → InstructionStep  (ปุ่ม ถัดไป)             :63-71
```

หน้าจอ **ไม่ตัดสินขั้นถัดไปเอง** (คอมเมนต์ที่ `:23`) แค่ส่ง action กลับ

ตอนที่ 3: ผู้ใช้กดปุ่ม → ส่ง action

1. กดปุ่ม เช่น "ใช่" (`StepView.tsx:51`) → `onAction({ type:'answer', value:'yes' })`
2. `handleAction` (`SessionPage.tsx:50-61`): กันกดซ้ำด้วย `isSendingRef` (`:51-52`) → `submit.mutate(action)` และเมื่อสำเร็จเพิ่มขั้นที่ผ่านลง `trail` (`:56`)
3. `useSubmitAction` (`queries.ts:125`) → `api/traversal.ts:58-66` → **`POST /traversal/sessions/:id/actions`**
4. body มี 4 แบบ ตามชนิดการกระทำ:
   - `{ "type":"answer", "value":"yes" | "no" }` (checkpoint)
   - `{ "type":"continue" }` (instruction ธรรมดา)
   - `{ "type":"confirm_safety" }` (instruction ที่ต้องยืนยันความปลอดภัย)
   - `{ "type":"input", "value":"E4" }` (กรอกข้อความ)

ตอนที่ 4: เซิร์ฟเวอร์ตัดสินขั้นถัดไป (Backend)

คำขอ → `cors` → `express.json` → `/traversal` → `traversal.routes.ts:32-36`

1. **auth middleware:** `optionalAuth` (ไม่มีบัตร = guest, บัตรเสีย = 401)
2. **ตรวจ input:** `parseTraversalAction` (`traversal.dto.ts:119`) `switch` ตาม `type`: `answer` ต้อง `yes`/`no`, `input` ต้องเป็นข้อความ ≤ 255 (`MAX_INPUT_LENGTH` `:5`) ชนิดอื่น → 400
3. **service** `TraversalService.submitAction` (`traversal.service.ts:86`):
   - `authorize` (`:136-141`) ถามว่า session นี้มีเจ้าของเป็น user ไหม (`SELECT user_id FROM sessions WHERE session_id = ? AND expires_at > NOW()`) ถ้ามีและไม่ใช่คนที่ขอ → 404

- `requireSession` (`:148`) → `SessionStore.find` (`session.store.ts:59-87`): อ่านแถว session + ประวัติขั้นที่ผ่านมา
- `requireGraph` (`:142`) หาผังจากหน่วยความจำ
- **`engineSubmitAction(session, graph, action)` (`:95`)** ← ตัดสินขั้นถัดไป
- `sessionStore.update(...)` (`:98-101`) บันทึกผล ← **รันหลัง engine อนุมัติเท่านั้น** ถ้า engine โยน error บรรทัดนี้ไม่ถูกรัน DB ไม่เปลี่ยน
- `toResponse` ประกอบคำตอบ

เครื่องสถานะ (`traversal-engine/traversal-engine.ts`)

`submitAction` (`:41-72`) → `resolveNextNode` (`:74-125`) ตามชนิดโหนดปัจจุบัน:

โหนด	ต้องส่ง	ไปโหนดไหน	ผิดชนิด
`checkpoint` `:80-88`	`answer`	`yes` → `on_yes` / `no` → `on_no`	`InvalidActionError`
`instruction` ธรรมดา `:98-101`	`continue`	`next`	`InvalidActionError`
`instruction` **ที่ `safety_critical`** `:91-97`	`confirm_safety`	`next`	ส่ง `continue` → **`SafetyConfirmationRequiredError`**
`input` `:104-119`	`input`	ตรง pattern → `next` เก็บค่าไว้ใน `variables[store_as]` · ไม่ตรง → `on_invalid` (หรือโหนดเดิม)	`InvalidActionError`
`resolution` / `escalation` `:121-123`	–	จบแล้ว	`InvalidActionError`

ถึงโหนดจบ → `status = 'completed'` (`:61`) แล้ว `renderNode` (`:126-148`) ประกอบข้อความ (แทน **`{{ตัวแปร}}`** ด้วยค่าที่ผู้ใช้กรอก `:168-172`), `reference = { source, pageRange }` (`:131-134`), `requiresSafetyConfirmation`, `isTerminal`, `outcomeKind`

ด่านความปลอดภัยบังคับสองฝั่ง: หน้าจอ (ติ๊กก่อนกด) **และ** engine (ไม่ยอมรับ `continue` ที่โหนด safety) ต่อให้เรียก API ตรงก็ข้ามไม่ได้

บันทึกผล (รายละเอียดอยู่ช่วง 6)

`SessionStore.update` (`session.store.ts:89-142`) ใน transaction เดียว:

```
UPDATE sessions SET current_node_id = ?, status = ?, variables = ?, expires_at = NOW() + INTERVAL ? HOUR WHERE session_id = ?
INSERT INTO session_history (session_id, step_order, node_id, action_type, action_value) VALUES (?, ?, ?, ?, ?)
```

ตอนที่ 5: ประกอบคำตอบ + AI ปรับถ้อยคำ

`toResponse` (`traversal.service.ts:154`) ได้ `{ sessionId, graphId, status, node, equipment, confidence, outcome, displayText }`

- `equipment`: จากหน่วยความจำ `findByCategory` (`:165`) ตามประเภทเครื่อง
- `displayText`: `displayTextOf` (`:172`) เรียก `RephraseService.rephrase` (`rephrase.service.ts:35-62`) → ข้ามโหนด safety/จบ (`:40`) · timeout 3 วินาที (`:46`) · ผลต้องผ่านด่านตรวจ `checkRephrase` (`rephrase-guard.ts:26-61`: ตัวเลข ซื่อรหัส คำ "ไม่" ความยาว) · ล้ม/ช้า/ไม่ผ่าน → `null` ใช้ข้อความต้นฉบับ prompt ที่ส่ง Groq อยู่ `groq-llm.client.ts:5-11` ห้ามเปลี่ยนความหมาย
- หน้าจอกันซ้ำอีกชั้น: ใช้ `displayText` เฉพาะเมื่อ `!safetyCritical && !requiresSafetyConfirmation && !isTerminal` (`SessionPage.tsx:123-125`) แล้วแสดงปุ่ม "ดูข้อความต้นฉบับ"

ตอบ **201** → `useCacheSession` ใส่ session ใหม่ในแคช (`onSuccess`) หน้าจอแสดงขั้นถัดไปทันที ถ้าเซิร์ฟเวอร์ปฏิเสธ `onSuccess` ไม่ทำงาน แคชยังเป็นสถานะเดิม หน้าจอไม่ขยับ (ตรงกับสถานะจริง)

ถ้ามีอะไรผิดพลาด

กรณี	ผลที่ได้
ส่ง `continue` ที่โหนด safety	400 `SAFETY_CONFIRMATION_REQUIRED` (ตาราง `ENGINE_ERRORS` `error-handler.ts:38-43`)
action ผิดชนิดกับโหนด	400 `INVALID_ACTION`
session จบแล้วยังส่ง action	409 `SESSION_COMPLETED`
session ไม่มี/หมดอายุ/ไม่ใช่ของเรา	404 `SESSION_NOT_FOUND`
ผังหายไป	404 `GRAPH_NOT_FOUND`
ส่ง action พร้อมกันสองคำขอ	ผู้แพ้อาจได้ 500 แทน 409 (UNIQUE `session_history (session_id, step_order)` ชนเป็น error ของ MySQL ที่ไม่ได้ห่อเป็น `ApiError`) ตามที่บันทึกไว้ ไม่ได้ทดสอบ

หน้าจอแสดง error ผ่าน `ErrorNotice` พร้อมปุ่ม "ดึงขั้นตอนล่าสุด" (`SessionPage.tsx:160`) ที่ล้างสถานะแล้วโหลด session ใหม่

ข้อสังเกต

- **session ของ guest ไม่ถูกตรวจเจ้าของ** ที่ `GET` / `actions` / `outcome`: `authorize` (`traversal.service.ts:136-141`) เช็กเฉพาะ session ที่มี `user_id` ถ้าเป็นของ guest (`user_id` เป็น NULL) ใครรู้ `sessionId` (UUID เดายาก) ก็อ่าน/เดินได้ มีแค่ `DELETE` และ `/history` ที่ตรวจ `client_id` (อ่านจากโค้ด ไม่ได้ทดสอบ)
- AI ถูกเรียก **ทุกครั้งที่ประกอบ response** (มีแคชผลที่ผ่านในหน่วยความจำ `rephrase.service.ts:27`, `:42-43`)
- ไม่มีขั้นที่ LLM ตัดสินว่าจะไปไหนต่อ ขั้นถัดไปมาจากผังล้วนๆ

คำถามเช็ก + เฉลย

1. สมชายกด "ใช่" สองครั้งเร็วๆ จะเกิดอะไร? เฉลย: หน้าจอกันไว้สองชั้น: `isSendingRef` ทิ้งคลิกที่สอง (`SessionPage.tsx:51-52`) และปุ่มถูกปิดระหว่างส่ง (`StepView.tsx:51-56` `disabled={isPending}`) ถ้าคำขอสองตัวหลุดไปถึง backend จริง ตัวหลังอาจชน UNIQUE ของ `session_history` แล้วได้ 500 (ข้อจำกัดที่รู้ ไม่ได้ทดสอบ)

2. ใครยิง API ตรงๆ ส่ง `{type:'continue'}` ที่โหนดความปลอดภัย จะเกิดอะไร? เฉลย: engine โยน `SafetyConfirmationRequiredError` (`traversal-engine.ts:91-94`) → `errorHandler` แปลงเป็น 400 `SAFETY_CONFIRMATION_REQUIRED` และ **ไม่มีอะไรถูกบันทึก** เพราะ `engineSubmitAction` (`traversal.service.ts:95`) รันก่อน `sessionStore.update` (`:98`)

3. ข้อความของโหนดแบบไหนที่ AI จะไม่ปรับถ้อยคำ? เฉลย: โหนดความปลอดภัย (`safetyCritical`/ต้องยืนยัน) และโหนดจบ (resolution/escalation) ทั้งฝั่ง backend (`rephrase.service.ts:40`) และหน้าจอ (`SessionPage.tsx:123`) เหตุผล: ด่านตรวจ (guard) ตรวจรูปแบบ ไม่ตรวจความหมาย เคยทดสอบแล้ว "ถอดปลั๊ก" → "เสียบปลั๊ก" ผ่านได้ ข้อความที่เสี่ยงสุดจึงไม่ให้ AI แตะ

4. ถ้า engine ปฏิเสธ action ข้อมูลใน DB เปลี่ยนไหม? เฉลย: ไม่เปลี่ยน `sessionStore.update` อยู่หลัง `engineSubmitAction` ถ้า engine โยน error บรรทัด update ไม่ถูกรัน ไม่ต้องมีโค้ด "ย้อนสถานะ"

5. ทำไมหน้าจอไม่ตัดสินขั้นถัดไปเอง ให้เซิร์ฟเวอร์ตัดสิน? เฉลย: กฎของระบบ ขั้นตอนและด่านความปลอดภัยต้องบังคับที่เดียวที่ยิง API ตรงก็ข้ามไม่ได้ (engine) หน้าจอแค่แสดงและส่ง action ถ้าให้หน้าจอตัดสินเอง คนแก้โค้ดหน้าจอหรือยิง API ข้ามไปจะข้ามด่านความปลอดภัยได้

ข้อเสนอ: `docs/USER_JOURNEY_WALKTHROUGH.md` ยังอ้างเลขบรรทัดเก่าของ `traversal.routes.ts`, `traversal.service.ts` และ `api/queries.ts` (ที่ถูกลบคอมเมนต์) ให้ผมรีเฟรชเลขทั้งเอกสารให้ตรงไฟล์ปัจจุบันไหมครับ และถ้าอยากได้ไฟล์บันทึกช่วงนี้ (`JOURNEY-5-SESSION.md` แบบเดียวกับช่วง 1–2) สั่งได้เลย พร้อมไป **ช่วงที่ 6 (บันทึกประวัติ)** เมื่อบอกครับ

ช่วงที่ 6: บันทึกประวัติ

ภาพรวม: ไม่มีปุ่ม "บันทึกประวัติ" ประวัติเกิดเองทุกครั้งที่ผู้ใช้กดตอบ ผู้ใช้มีแค่ปุ่มเสริม 2 อย่าง คือ "บันทึกผล" (ไม่บังคับ) และ "ยกเลิกการตรวจ" และ **ไม่มีตาราง chat/message/references** มีแค่ `sessions` กับ `session_history`

ก. บันทึกอัตโนมัติทุกขั้น (เกิดตอนผู้ใช้กดตอบ)

1. ผู้ใช้กด เช่น "ใช่" → `POST /traversal/sessions/:id/actions` (รายละเอียดช่วง 5)
2. `TraversalService.submitAction` (`backend/src/traversal/traversal.service.ts:86`):
   - ตรวจเจ้าของ `authorize` (`:91`) → อ่าน session จาก DB `requireSession` (`:92`) → หาผังจากหน่วยความจำ `requireGraph` (`:93`)
   - ให้ engine ตัดสินขั้นถัดไป `engineSubmitAction(session, graph, action)` (`:95`)
   - **บันทึกผลลง DB** `sessionStore.update(...)` (`:98-101`) ← รันหลัง engine อนุมัติเท่านั้น ถ้า engine โยน error บรรทัดนี้ไม่ถูกรัน DB ไม่เปลี่ยน
   - ประกอบคำตอบ `toResponse` (`:103`) แล้ว route ตอบ 201 (`traversal.routes.ts:34`)
3. `SessionStore.update` (`backend/src/traversal/session.store.ts:89-142`) ทำใน **transaction เดียว** (สำเร็จทั้งคู่หรือย้อนกลับทั้งคู่):

```
-- :106-112  ย้ายพิกัดไปโหนดใหม่ + ต่ออายุ
UPDATE sessions SET current_node_id = ?, status = ?, variables = ?, expires_at = NOW() + INTERVAL
? HOUR
 WHERE session_id = ?
-- :122-125  จดว่าผู้ใช้ทำอะไรที่โหนดไหน (1 แถวต่อขั้น)
INSERT INTO session_history (session_id, step_order, node_id, action_type, action_value)
VALUES (?, ?, ?, ?, ?)
```

- `step_order` = ลำดับขั้น (นับจาก 0) · `node_id` = โหนดที่ผู้ใช้อยู่ตอนกด · `action_type` ∈ answer/continue/confirm_safety/input · `action_value` = yes/no หรือข้อความที่กรอก
- **อายุข้อมูล** (`session.store.ts:8-10`, `:99-100`): กำลังเดิน 24 ชั่วโมง (ต่ออายุทุกขั้น) · ถึงโหนดจบ (`completed`) 90 วัน
- `UNIQUE (session_id, step_order)` (`backend/migrations/004_sessions.sql`) กันสองคำขอซ้อนกัน ผู้แพ้ชนข้อจำกัดนี้

ข. บันทึกข้อความผลลัพธ์ (ปุ่มเสริม ไม่บังคับ)

เรื่อง: ถึงโหนดจบแล้ว สมชายพิมพ์ "ทำตามแล้วหาย" แล้วกด "บันทึกผล"

1. ฟอร์มแสดงเฉพาะที่โหนดจบ: `frontend/src/components/step/StepView.tsx:27-29` เห็น `node.isTerminal` → `OutcomeStep` ใส่ฟอร์มไว้ที่ `OutcomeStep.tsx:44`
2. กรอกข้อความ ปุ่มจะกดได้เมื่อ `checkOutcome` ได้ `ok` (ไม่ว่าง และไม่เกิน 1,000 ตัว `lib/outcome.ts:9-14`) `components/OutcomeForm.tsx:43-50` → `handleSubmit` ส่ง `normalizeOutcome(value)` (ตัดช่องว่างหัวท้าย)

3. กดปุ่ม "บันทึกผล" (`OutcomeForm.tsx:83-85`) → `handleOutcome` (`pages/SessionPage.tsx:63-72`) กันกดซ้ำด้วย `isSavingOutcomeRef` แล้ว `saveOutcome.mutate(text)`
4. `useSubmitOutcome` (`api/queries.ts:132`) → `api/traversal.ts:64-69` → **`POST /traversal/sessions/:id/outcome`** body `{ "text": "ทำตามแล้วหาย" }`
5. Backend: `traversal.routes.ts:38-42` ผ่าน `optionalAuth` แล้วเรียก `parseOutcomeBody` (`traversal.dto.ts:166-185`): ต้องเป็นข้อความ, `trim` แล้วไม่ว่าง, ไม่เกิน 1,000 (`MAX_OUTCOME_LENGTH` `:6`) ผิด → 400 `INVALID_OUTCOME`
6. `TraversalService.submitOutcome` (`traversal.service.ts:114`):
   - ตรวจเจ้าของ/หา session/หาผัง (`:119-121`) ไม่ใช่ของเรา/ไม่มี → 404
   - **ต้องจบแล้วเท่านั้น** (`:123-125`) ยัง `in_progress` → 409 `SESSION_NOT_COMPLETED`
   - บันทึก `sessionStore.saveOutcome` (`:127`) ได้ `false` (เคยบันทึกแล้ว) → 409 `OUTCOME_ALREADY_SUBMITTED` (`:128-130`)
7. `SessionStore.saveOutcome` (`session.store.ts:171-182`):

```
UPDATE sessions SET outcome_text = ?, outcome_at = NOW()
 WHERE session_id = ? AND status = 'completed' AND outcome_text IS NULL
```

เงื่อนไขใน `WHERE` คือประตูในคำสั่งเดียว: ต้องจบแล้ว **และ** ยังไม่เคยบันทึก (atomic) สองคำขอพร้อมกัน ชนะได้คนเดียว ผู้แพ้ `affectedRows = 0` → 409 8. ตอบ 200 (`traversal.service.ts:132-133` `toResponse(session, node, graph, text)`) → หน้าจอจดลงแคช → `OutcomeForm.tsx:32-41` `isSaved` เปลี่ยนเป็น "บันทึกผลแล้ว" พร้อมข้อความอ่านอย่างเดียว 9. ข้อควรรู้: **บันทึกได้ครั้งเดียว แก้ไขภายหลังไม่ได้** (ข้อความบนหน้าจอ `OutcomeForm.tsx:58-60`) · ผลลัพธ์**ไม่เข้า engine** ไม่เปลี่ยนสถานะ · กด F5 แล้วยังเห็น เพราะ `GET session` อ่าน `outcome_text` กลับมา (`findOutcome` `session.store.ts:184-192`)

ค. ยกเลิกการตรวจ (ปุ่มเสริม)

1. ปุ่ม "ยกเลิกการตรวจ" แสดงเฉพาะตอนยังไม่จบ: `components/SessionHeader.tsx:28-32` (`canAbandon={!node.isTerminal}` `SessionPage.tsx:151`)
2. `handleAbandon` (`SessionPage.tsx:74-76`) → `useAbandonSession` (`api/queries.ts:139` สำเร็จแล้วลบแคชของ session นั้น กันกดย้อนกลับแล้วเห็นขั้นเก่า) → `api/traversal.ts:71-75` **`DELETE /traversal/sessions/:id`** แนบ `ownerHeaders()`
3. Backend: `traversal.routes.ts:44-49` `optionalAuth` → `resolveRequiredOwner` (ล็อกอิน = user, ไม่ล็อกอิน = `X-Client-Id` บังคับ) → `TraversalService.abandonSession` (`traversal.service.ts:107-110`) ลบไม่ได้ → 404 `SESSION_NOT_FOUND` → ตอบ **204**
4. `SessionStore.delete` (`session.store.ts:161-169`):

```
DELETE FROM sessions WHERE session_id = ? AND ${ownerWhere} AND expires_at > NOW()
```

`session_history` และ `session_feedback` ถูกลบตามด้วย `ON DELETE CASCADE` ผู้ที่ไม่ใช่เจ้าของลบไม่ได้ (404 เหมือนไม่มีอยู่)

ข้อสังเกตช่วง 6

- เจ้าของ session: ผู้ล็อกอิน → `user_id` · guest → `client_id` · ไม่ส่งอะไรเลย → ไม่มีเจ้าของ **จะไม่ปรากฏในประวัติใคร** (`backend/src/auth/session-owner.ts:14-23`)
- ไม่มีงานลบแถวที่หมดอายุ (ค้น cron/job ลบแล้วไม่พบ) แถวหมดอายุแค่ถูกกรองออกตอนอ่าน
- ไม่มี endpoint ย้อนขั้นหรือแก้คำตอบ ผิดทางต้อง "เริ่มอาการนี้ใหม่" (สร้าง session ใหม่)

เฉลยคำถามช่วง 6

- **กดบันทึกผลตอนยังไม่ถึงโหนดจบ (ยิง API ตรง):** 409 `SESSION_NOT_COMPLETED` (`traversal.service.ts:123-125`) และ SQL มี `AND status = 'completed'` เป็นประตูอีกชั้น
- **สองแท็บกดบันทึกผลพร้อมกัน:** `UPDATE ... WHERE outcome_text IS NULL` คำสั่งเดียว คนแรก `affectedRows = 1` คนที่สอง 0 → 409
- **ยกเลิกแล้วอะไรหาย:** แถว `sessions` + `session_history` + `session_feedback` (CASCADE) ไม่เหลือใน DB สถิติ admin จึงนับ "ยกเลิก" ไม่ได้ มีแต่ "เดินค้างแล้วหมดอายุ"
- **ปิดแท็บหลังตอบครบแต่ยังไม่พิมพ์ผลลัพธ์:** ประวัติไม่หาย session `completed` บันทึกครบทุกขั้นแล้ว เก็บ 90 วัน ข้อความผลลัพธ์เป็นแค่ส่วนเสริม

โน้ตของฉัน (ช่วง 6)

-

ช่วงที่ 7: หน้าประวัติการซ่อม (รายการ + รายละเอียด)

ภาพรวม: สมชายตรวจจบหลายครั้งแล้ว กดเมนู "ประวัติการซ่อม" หน้าแรกคือ **รายการ** กดแถวแล้วเข้า **รายละเอียด** คำถามหลักสองข้อ: ระบบรู้ได้ไงว่านี่คือประวัติของสมชาย และถ้าผังถูกแก้ไปแล้วจะโชว์ประวัติเก่าอย่างไร

ก. หน้ารายการ (`/history`)

1. กดเมนู "ประวัติการซ่อม" (`components/layout/AppShell.tsx:30`) → `App.tsx:45` แสดง `HistoryPage`
2. `pages/HistoryPage.tsx:24-28` อ่านเลขหน้าจาก URL `?page=` (ไม่ใช่เลขบวก → หน้า 1) แล้ว `:32` เรียก `useHistory(page)` (`api/queries.ts:58`)
   - key ของแคชมี identity: `['history', identity, 'list', page]` และใช้ `keepPreviousData` (เปลี่ยนหน้าแล้วรายการเดิมค้างจนผลใหม่มา ไม่กะพริบ)
3. `api/history.ts:11-13` `listHistory` → **`GET /history?page=N`** พร้อม `headers: ownerHeaders()` (`lib/clientHeaders.ts:12-14`): ล็อกอินอยู่ → ไม่ส่งอะไร (token แนบโดย `api/http.ts`) · guest → ส่ง `X-Client-Id` นี่คือสิ่งที่บอก backend ว่า "ฉันคือเจ้าของประวัติชุดไหน"
4. Backend: `backend/src/history/history.routes.ts:13-17`
   - `optionalAuth` อ่านบัตร
   - `resolveRequiredOwner(currentUser(request), request.headers['x-client-id'])` (`auth/session-owner.ts:7-12`): ล็อกอิน → `{kind:'user', userId}` · ไม่ล็อกอิน → ต้องมี `X-Client-Id` ไม่งั้น 400 `INVALID_CLIENT_ID`
   - `parseHistoryQuery(request.query)` (`history/history.dto.ts:41`): `page` ≥ 1 ค่าเริ่มต้น 1, `pageSize` 1–50 ค่าเริ่มต้น 10 (`:6-7`) ผิด → 400 `INVALID_HISTORY_QUERY`
5. `HistoryService.list` (`history/history.service.ts:45`) เรียก `HistoryStore.listCompleted(owner)` (`history/history.store.ts:48-60`):

```
-- :50-57  ownerWhere = "(user_id = ?)" หรือ "(client_id = ? AND user_id IS NULL)"
SELECT session_id, graph_id, current_node_id, UNIX_TIMESTAMP(created_at) AS created_epoch
  FROM sessions
 WHERE ${where.sql} AND status = 'completed' AND expires_at > NOW()
 ORDER BY created_at DESC
 LIMIT ?          -- MAX_HISTORY_ROWS = 50 (history.store.ts:10)
```

"ประวัติของสมชาย" ถูกกรองที่ SQL ด้วย `ownerWhere` (`session-owner.ts:25-30`) เห็นเฉพาะ session ที่ เป็นของเจ้าของนี้ + จบแล้ว + ยังไม่หมดอายุ 6. กลับมาที่ service: เรียงใหม่สุดก่อน → แบ่งหน้าในหน่วยความจำ (หน้าเกินถูกดึงกลับหน้าสุดท้าย) → `toListItem` (`:110`) ต่อชื่ออาการ/รุ่นจากผังในหน่วยความจำ (ผังหายไป → `graph: null`, `outcomeKind: null` รายการไม่ล้ม) 7. ตอบ:

```
{ "items": [ { "sessionId": "...", "graphId": "...", "startedAt": "2026-10-22T03:00:00.000Z",
               "outcomeKind": "user_fixed", "graph": { "manualId":"...", "entrySymptom":"...",
"deviceCategory":"microwave", "brand":"Samsung", "modelPattern":"..." } } ],
  "total": 12, "page": 1, "pageSize": 10, "totalPages": 2 }
```

8. หน้าจอแสดงแต่ละแถวที่ `components/history/HistoryRow.tsx:23-78` (ชื่ออาการ, ประเภท/ยี่ห้อ/รุ่น, วันเวลา, ป้ายสถานะ) เปลี่ยนหน้าด้วย `Pagination` (`HistoryPage.tsx:110`) ถ้าไม่มีประวัติ แสดง "ยังไม่มีประวัติการซ่อม" พร้อมลิงก์เลือกเครื่อง (`:87-97`)
9. หมายเหตุ: รายการดูแค่ "โหนดสุดท้ายเป็นจบแบบไหน" (`getOutcomeKind`) ไม่ replay ทุกแถว เพราะหนัก

ข. หน้ารายละเอียด (`/history/:id`)

1. กดแถว (`HistoryRow.tsx:32-35` `Link to="/history/:id"`) → `App.tsx:46` แสดง `HistoryDetailPage`
2. `pages/HistoryDetailPage.tsx:52` เรียก `useHistoryDetail(sessionId)` (`api/queries.ts:67`) → `api/history.ts:16-18` **`GET /history/:id`** + `ownerHeaders()`
3. Backend: `history.routes.ts:20-24` (`optionalAuth` + `resolveRequiredOwner`) → `HistoryService.detail` (`history.service.ts:68`) ทำตามลำดับ:
   - **`requireOwned`** (`:100`) → `HistoryStore.findCompleted` (`history.store.ts:62-82`): ตรวจเจ้าของ + จบแล้ว + ไม่หมดอายุในคำสั่งเดียว ไม่ใช่ของเรา → 404 `HISTORY_NOT_FOUND` (ไม่ใช่ 403 เพื่อไม่บอกว่า id นั้นมีจริง `history.service.ts:35-36`)
   - `findFeedback` (`history.store.ts:114-123`) อ่านคะแนนที่เคยให้
   - `graphRepository.findById(graphId)` หาผังจากหน่วยความจำ
   - `findSteps` (`history.store.ts:84-93`) อ่านขั้นที่เคยผ่าน
   - `buildDetail` (`history.service.ts:126`) ประกอบรายละเอียด

```
-- findCompleted  history.store.ts:64-71
SELECT session_id, graph_id, current_node_id, variables, confidence, model_selection,
outcome_text,
       UNIX_TIMESTAMP(created_at) AS created_epoch
  FROM sessions WHERE session_id = ? AND ${where.sql} AND status = 'completed' AND expires_at >
NOW()
-- findSteps  :85-91
SELECT node_id, action_type, action_value FROM session_history WHERE session_id = ? ORDER BY
step_order
-- findFeedback  :115-119
SELECT helpful, `comment` FROM session_feedback WHERE session_id = ?
```

⚠️ `findSteps` ไม่ตรวจเจ้าของเอง ต้องเรียกหลัง `findCompleted` ผ่านเท่านั้น (service ทำถูกแล้ว) 4. **ปัญหาใหญ่: ผังถูกแก้หลังผู้ใช้ตรวจจบ** ประวัติเก็บแค่ "รหัสโหนด + สิ่งที่กด" ไม่ได้เก็บข้อความตอนนั้น อ่านย้อนหลังต้องเอาข้อความจากผัง **ปัจจุบัน** ถ้าผังเปลี่ยน อาจโชว์ขั้นเก่าคู่ข้อสรุปใหม่ที่ไม่เกี่ยวกัน ทางออกคือ **เล่นซ้ำเพื่อพิสูจน์** (`replayReaches` `history.service.ts:232`):

- เริ่มที่ `entry_node` ของผังปัจจุบัน → ป้อน action ที่บันทึกไว้ทีละขั้นผ่าน engine จริง
- ต้องผ่านทุกขั้นและไปจบที่ "โหนดสุดท้ายเดิม" ในสถานะ `completed` → ผ่าน: ตอบข้อมูลเต็ม `available: true`
- ไม่ผ่าน (โหนดหาย, ทางเดินเปลี่ยน, engine ไม่รับ action เดิม) → ตอบ `available: false` (HTTP 200 ไม่ใช่ error) ยังมีข้อความที่ผู้ใช้บันทึกเอง คะแนนที่ให้ และปุ่มลบ แต่ไม่มี steps/finalNode/คำเตือน (ไม่ปล่อยข้อมูลครึ่งๆ กลางๆ)

5. ตอบ `HistoryDetail`: `available`, `steps[]`, `finalNode`, `safetyWarnings[]`, `reference`, `outcome`, `feedback`, `confidence`, `modelSelection` (shape ที่ `frontend/src/api/types.ts`)
6. หน้าจอแสดงตามลำดับ: กล่อง "ผังถูกเปลี่ยน" ถ้า `available:false` (`HistoryDetailPage.tsx:115-119`) → คำเตือนความปลอดภัย (`:122-130`, เฉพาะที่ผังมีจริง) → ผลการตรวจ + ที่มา (`:132-149`) → ความมั่นใจ (`:151-163` `null` = "ไม่มีคะแนน" ไม่แสดงเป็น 0) → ขั้นตอนที่ผ่านมา (`:165-183`) → ผลที่ผู้ใช้บันทึก (`:185-190`) → ฟอร์มให้คะแนน (`:192-199` ช่วง 8) → ปุ่ม "ตรวจอาการนี้อีกครั้ง" (`:205-207`, `handleRestart :88-91` เริ่ม session ใหม่ด้วยรุ่นเดิม) → ปุ่มลบ (`:226-228`)

ค. ลบจากประวัติ

1. กด "ลบ" (`HistoryRow.tsx:59-61` หรือ `HistoryDetailPage.tsx:226-228`) → `DeleteConfirm` ขึ้นขั้นยืนยัน → `useDeleteHistory` (`api/queries.ts:150`) → `api/history.ts:21-26` **`DELETE /history/:id`**
2. Backend: `history.routes.ts:34-39` → `HistoryService.remove` (`history.service.ts:78`) → `HistoryStore.deleteCompleted` (`history.store.ts:95-103`):

```
DELETE FROM sessions WHERE session_id = ? AND ${where.sql} AND status = 'completed' AND expires_at > NOW()
```

3. ตอบ 204 · ไม่ใช่ของเรา/ไม่พบ → 404 · `session_history` กับ `session_feedback` ถูกลบตาม (CASCADE) · ฝั่งหน้าจอลบแคชของรายการนั้นแล้วโหลดรายการใหม่ทุกหน้า (`onSettled`)

ข้อสังเกตช่วง 7

- เพดาน **50 รายการต่อผู้ใช้** (`MAX_HISTORY_ROWS`) รายการเก่ากว่านั้นเปิดดู/ลบจากหน้ารายการไม่ได้
- guest ผูกกับ `client_id` ใน localStorage: ล้างข้อมูลเบราว์เซอร์ = ประวัติหาย และใครรู้ id ก็เปิดประวัติ guest ของ id นั้นได้ (ไม่ใช่การยืนยันตัวตน)
- ประวัติก่อนล็อกอินไม่ย้ายเข้าบัญชี (หน้าจอบอกชัด `HistoryPage.tsx:74-76`)

เฉลยคำถามช่วง 7

- **ระบบรู้ได้ไงว่า session ไหนเป็นของสมชาย:** จาก `sessions.user_id` (ล็อกอิน) หรือ `sessions.client_id` (guest) ที่เก็บตอนเริ่มตรวจ (`session.store.ts:38-57`) และทุก query ของประวัติมี `ownerWhere` ใน SQL (`history.store.ts:54, :69, :99`) ตัวระบุเจ้าของมาจาก token หรือหัว `X-Client-Id`
- **เดา session id ของคนอื่นถูก:** ได้ 404 `HISTORY_NOT_FOUND` เพราะกรองด้วย `ownerWhere` แล้วผลว่างเหมือนไม่มี (ไม่ตอบ 403)
- **admin แก้ผังหลังตรวจจบ:** `replayReaches` เล่นซ้ำกับผังปัจจุบัน ไม่ไปจบที่โหนดเดิม → `available:false` (HTTP 200) หน้าจอบอกว่าผังถูกเปลี่ยน ไม่โชว์ขั้นเก่าคู่ข้อสรุปใหม่
- **ทำไมรายการไม่ replay ทุกแถว:** ต้องดึงขั้นของแต่ละ session แยกซึ่งหนัก ตัวตรวจเข้มอยู่ที่หน้ารายละเอียด
- **มีประวัติ 60 รายการ เห็นกี่รายการ:** สูงสุด 50 ล่าสุด (`LIMIT` ใน SQL) และ `total` ก็นับจากที่ดึงมา 50

โน้ตของฉัน (ช่วง 7)

-

ช่วงที่ 8: ให้คะแนน 👍/👎 + ข้อความ

ภาพรวม: สมชายเปิดรายละเอียดประวัติ กด 👍 พิมพ์ "ช่วยได้มาก" แล้วบันทึก คะแนนถูกเก็บในตาราง `session_feedback` (1 session = 1 แถว ส่งซ้ำ = แก้ค่าเดิม) และผู้ดูแลอ่านไปทำสถิติในช่วง 9 **ให้คะแนนได้เฉพาะในหน้ารายละเอียดประวัติ** ไม่มีฟอร์มที่หน้าจบการตรวจ (หน้าจบมีแต่ฟอร์ม "ผลเป็นอย่างไรบ้าง")

ก. ฝั่งหน้าจอ

1. ฟอร์มอยู่ใน `components/history/FeedbackButtons.tsx` ที่หน้ารายละเอียดเรียกใช้ที่ `pages/HistoryDetailPage.tsx:192-199` แสดงเฉพาะเมื่อ `data.available` (ผังยังตรงเดิม)
2. ค่าเริ่มต้นของฟอร์มมาจากคะแนนที่เคยบันทึกไว้ (`saved`): `useState(saved?.helpful ?? null)` และ `useState(saved?.comment ?? '')` (`FeedbackButtons.tsx:28-29`) จึงเปิดมาแล้วเห็นของเดิม และส่งซ้ำคือแก้ค่าเดิม
3. ปุ่ม 👍 "ช่วยได้" (`:45-47`) และ 👎 "ไม่ช่วย" (`:48-50`) แค่ตั้งค่า `helpful` ใน state ยังไม่ส่ง · ต้องเลือกหนึ่งอย่าง (บังคับ)
4. ช่องข้อความ (`:62-71`): ไม่บังคับ ไม่เกิน 500 ตัวอักษร (`MAX_FEEDBACK_COMMENT_LENGTH` `lib/textInput.ts:23`) มีตัวนับ `formatCommentCounter` (`textInput.ts:35`) และข้อความเตือน "ไม่ต้องใส่ข้อมูลส่วนตัว" (`:57-59`)
5. ปุ่ม "บันทึก" (`:78-80`) กดได้เมื่อ (`:31-32`): เลือก 👍/👎 แล้ว **และ** ข้อความไม่เกิน 500 (`checkComment` `lib/textInput.ts:31`) **และ** ไม่กำลังส่ง **และ** ค่าต่างจากที่บันทึกไว้ (`isSameFeedback` `lib/textInput.ts:44` ถ้าเหมือนเดิมแสดง "บันทึกแล้ว" และปุ่มถูกปิด)
6. กดบันทึก → `handleSubmit` (`FeedbackButtons.tsx:34-38`) → `onSubmit(toFeedback(helpful, comment))` (`lib/textInput.ts:39`): ตัดช่องว่างหัวท้าย และ **ข้อความว่างส่งเป็น `null`** ให้ตรงกับที่ DB เก็บ (NULL ไม่ใช่สตริงว่าง) → `HistoryDetailPage.tsx:195` `feedback.mutate(value)`
7. `useSubmitFeedback(sessionId)` (`api/queries.ts:166`) → `api/history.ts:29-35` `submitFeedback` → **`POST /history/:id/feedback`** body `{ "helpful": true, "comment": "ช่วยได้มาก" }` (comment อาจเป็น `null`) + `headers: ownerHeaders()`

ข. ฝั่งเซิร์ฟเวอร์

1. คำขอผ่าน `cors` → `express.json` → router `/history` → `backend/src/history/history.routes.ts:27-31`
2. `optionalAuth` อ่านบัตร → `resolveRequiredOwner(currentUser(request), request.headers['x-client-id'])` ระบุเจ้าของ (ล็อกอิน = user, ไม่ล็อกอิน = `X-Client-Id` บังคับ)
3. `parseFeedbackBody` (`history/history.dto.ts:112-135`) ตรวจ body:
   - ต้องเป็น JSON object
   - `helpful` **ต้องเป็น boolean จริง** ไม่รับข้อความ `"true"` หรือเลข 1 → ไม่งั้น 400 `INVALID_FEEDBACK`
   - `comment` ไม่บังคับ: ไม่ส่ง/`null` → `null` · ไม่ใช่ข้อความ → 400 · ตัดช่องว่างแล้วเกิน 500 → 400 · ว่างหลังตัด → `null`
4. ตอบ 200 (ไม่ใช่ 201 เพราะส่งซ้ำ = แก้ค่าเดิม) `response.status(200).json(await history.saveFeedback(owner, id, parseFeedbackBody(body)))` (`history.routes.ts:29`)

5. `HistoryService.saveFeedback` (`history/history.service.ts:86-94`):
    - **`requireOwned`** (`:91`) session ต้องเป็นของผู้ขอ และจบแล้ว ไม่งั้น 404 `HISTORY_NOT_FOUND` (กันให้คะแนน session ของคนอื่น)
    - `store.saveFeedback(sessionId, body)` (`:92`)
    - คืน `{ helpful, comment }` ที่บันทึก (`:93`)
6. `HistoryStore.saveFeedback` (`history/history.store.ts:105-112`):

```
INSERT INTO session_feedback (session_id, helpful, `comment`)
VALUES (?, ?, ?)
ON DUPLICATE KEY UPDATE helpful = VALUES(helpful), `comment` = VALUES(`comment`)
```

`helpful` เก็บเป็น 1/0 (`feedback.helpful ? 1 : 0` `:110`) · `ON DUPLICATE KEY UPDATE` = ถ้ามีแถวของ session นี้อยู่แล้ว ให้แก้ค่าเดิมแทนเพิ่มแถวที่สอง (คำสั่งเดียว atomic ไม่ต้องตรวจก่อนแล้วค่อยเขียน)

ค. ฐานข้อมูล

ตาราง `session_feedback` (`backend/migrations/005_session_feedback.sql:10-22`):

- `session_id` เป็น **PRIMARY KEY** และ FK ไป `sessions` (`ON DELETE CASCADE`) → 1 session มีได้ 1 แถว · ลบ session แถวนี้หายตาม
- `helpful TINYINT(1)` (1 = 👍, 0 = 👎) · `comment TEXT NULL` · `created_at`, `updated_at` (มี `ON UPDATE CURRENT_TIMESTAMP` แก้แล้วเวลาแก้ไขขยับ)

ง. ตอบกลับและหน้าจออัปเดต

1. ตอบ:

```
{ "helpful": true, "comment": "ช่วยได้มาก" }
```

2. `useSubmitFeedback` `onSuccess` เอาค่าที่เซิร์ฟเวอร์ตอบมาใส่ในแคชรายละเอียดทันที (`setQueryData` ใน `api/queries.ts:166`) ฟอร์มจึงรู้ว่า "บันทึกแล้ว" โดยไม่ต้องโหลดซ้ำ → `isSameFeedback` เป็นจริง → ขึ้น "บันทึกแล้ว" (`FeedbackButtons.tsx:81-85`) และปุ่มบันทึกถูกปิด
3. ผิดพลาด (เช่น 400/404/เครือข่าย) แสดง `ErrorNotice` ใต้ฟอร์ม (`:75`)

ข้อสังเกตช่วง 8

- ให้คะแนนได้เฉพาะหลังเข้าหน้ารายละเอียดประวัติ ผู้ใช้ต้องกดเข้า `/history/:id` เอง
- ไม่มีผลต่อขั้นตอนหรือการตัดสินใดของระบบ เก็บไว้ประเมินเท่านั้น (คอมเมนต์ `FeedbackButtons.tsx:19-22`)
- ความเห็นเป็นข้อความอิสระ อาจมีข้อมูลส่วนตัวปน หน้าจอเตือนผู้ใช้ (`:57-59`) และหน้าแอดมินแสดงตามที่พิมพ์
- ตรวจ 2 ที่ (หน้าจอบอกทันที / backend ตัดสิน) เพดาน 500 คัดลอกมือ ต้องแก้สองที่เมื่อเปลี่ยน

เฉลยคำถามช่วง 8

- **ส่งคะแนนซ้ำเปลี่ยนจาก 👍 เป็น 👎 เกิดอะไร:** `ON DUPLICATE KEY UPDATE` แก้แถวเดิม ไม่เพิ่มแถวใหม่ (`session_id` เป็น PRIMARY KEY) `updated_at` ขยับ
- **ส่ง `{"helpful": "true"}` (ข้อความ):** 400 `INVALID_FEEDBACK` เพราะต้องเป็น boolean จริง (`history.dto.ts` ตรวจ `typeof obj.helpful !== 'boolean'`)
- **ให้คะแนน session ของคนอื่นโดยเดา id:** 404 `HISTORY_NOT_FOUND` จาก `requireOwned` (`history.service.ts:91`) ก่อนถึงขั้นเขียน DB
- **ไม่พิมพ์ข้อความ:** ไม่เป็นไร ส่ง `comment: null` DB เก็บ NULL ไม่ใช่สตริงว่าง (`toFeedback`)
- **ส่งข้อความ `" "` (ช่องว่างล้วน):** ตัดแล้วว่าง → `null`

โน้ตของฉัน (ช่วง 8)

-

ช่วงที่ 9: แดชบอร์ดผู้ดูแล (สถิติ + ความเห็น)

ภาพรวม: ผู้ดูแลระบบ (admin) เปิด `/admin/analytics` ดูสถิติ N วันล่าสุด และ `/admin/feedback` อ่านรายการความเห็นที่ผู้ใช้ให้ไว้ในช่วง 8 อ่านอย่างเดียว ไม่มีการแก้ข้อมูล และ **ไม่มีแดชบอร์ดฝั่งผู้ใช้ทั่วไป**

ก. สถิติ `/admin/analytics`

ฝั่งหน้าจอ

1. เมนู "ผู้ดูแลระบบ" แสดงเฉพาะ `role === 'admin'` (`components/layout/AppShell.tsx:38, :46`) · `/admin` พาไป `/admin/analytics` (`App.tsx:50`) และครอบด้วย `AdminRoute` (`App.tsx:52-58`)
2. `components/auth/AdminRoute.tsx:19-40` ตรวจสถานะ: กำลังยืนยัน token → "กำลังตรวจสอบสิทธิ์" · guest → พาไปล็อกอิน (`?reason=login-required&next=...`) · ล็อกอินแต่ไม่ใช่ admin → `ForbiddenPage` · admin → แสดงหน้า **นี่เป็น UX เท่านั้น** ตัวบังคับจริงคือ backend (หัวไฟล์ `:16-17`)
3. หน้า `pages/AdminAnalyticsPage.tsx` อ่านช่วงวันจาก URL (`?days=7|30|90` ค่าเริ่มต้น 30 `lib/adminParams.ts`) ปุ่มเลือกช่วง (`:47-57`) → `setSearchParams`
4. `useAdminSummary(days)` (`api/queries.ts:81`) → `api/admin.ts:23-25` **`GET /admin/analytics/summary?days=30`** (แนบ `Authorization: Bearer ...` โดย `http.ts` ไม่ส่ง `X-Client-Id` เพราะไม่ใช่เรื่อง session) key ของแคชมี identity (`['admin', identity, 'summary', days]`)
5. แสดงผลจาก `AdminSummary` ตรงๆ ไม่คำนวณเอง: การ์ดตัวเลข (`:83-103`) การตรวจทั้งหมด/จบแล้ว/กำลังเดิน/เดินค้างแล้วหมดอายุ · **การ์ด "ความเห็น 👍 / 👎"** (`:92-96`) และ **"อัตราความพอใจ"** (`:97`) ที่ใช้ข้อมูลจากช่วง 8 · ความมั่นใจเฉลี่ย (`:98-102`) · แยกตามเจ้าของ (`:105-116`) · กราฟรายวัน `DailyBars` (`:118`) · `BarList` ผลการตรวจ/ประเภทเครื่อง/อาการยอดนิยม (`:120-142`) ค่า `null` (อัตราพอใจ, ความมั่นใจเฉลี่ย) แสดงเป็น "-" ไม่ใช่ 0

ฝั่งเซิร์ฟเวอร์

1. คำขอ → `cors` → `express.json` → router `/admin` (`app.ts:75`) → `backend/src/admin/admin.routes.ts:34-41`
2. **ด่านสิทธิ์ก่อนเสมอ:** `requireAuth` (`:36`) → `requireAdmin` (`:37`) (ตรวจสิทธิ์ก่อน parse query) ไม่มี/บัตรเสีย → 401 · ล็อกอินแล้วไม่ใช่ admin → 403 `FORBIDDEN` (`auth/auth.middleware.ts:59-80`) ติดทีละ route ไม่ครอบทั้ง path
3. ตรวจ query: `parseSummaryQuery` (`admin/admin.dto.ts:140`): `days` 1–90 ค่าเริ่มต้น 30 ผิด → 400 `INVALID_ADMIN_QUERY`
4. `AdminService.summary` (`admin/admin.service.ts:52`):
   - คิดช่วงเวลาตามเวลาไทย: เลขวันนี้ (`thaiDayIndex`) ย้อน `days-1` วัน แล้วแปลงเป็น epoch (`:53-55`)
   - **ยิง 5 query พร้อมกัน** ด้วย `Promise.all` (`:57-63`): `sessionTotals`, `feedbackTotals`, `graphCounts`, `completedEndNodes`, `perDay`
   - ประกอบผล: นับผลลัพธ์ตามชนิด (`:66-77`), อันดับอาการ/ประเภทเครื่อง ตัดเหลือ `TOP_LIMIT` (`:80-100`), เติมวันที่ไม่มีข้อมูลเป็น 0 (`:103-108`), อัตราความพอใจ `helpful / total` หรือ `null` ถ้า `total === 0` (`:123`)

5. `AdminStore` (`admin/admin.store.ts`) SQL อ่านอย่างเดียว ทุกคำสั่งใช้ช่วงเวลา `window` เดียวกัน:

```
-- feedbackTotals  :140-148   ← ข้อมูลจากช่วง 8 เข้ามาที่นี่
SELECT COUNT(*) AS total, SUM(helpful = 1) AS helpful_count
  FROM session_feedback
 WHERE UNIX_TIMESTAMP(created_at) >= ? AND UNIX_TIMESTAMP(created_at) < ?
-- sessionTotals  :111-138   (นับหลายแบบในคำสั่งเดียวด้วย SUM(เงื่อนไข))
SELECT COUNT(*) AS total,
       SUM(status = 'completed') AS completed,
       SUM(status = 'in_progress' AND expires_at > NOW()) AS in_progress,
       SUM(status = 'in_progress' AND expires_at <= NOW()) AS expired_unfinished,
       SUM(user_id IS NOT NULL) AS by_user,
       SUM(user_id IS NULL AND client_id IS NOT NULL) AS by_guest,
       SUM(user_id IS NULL AND client_id IS NULL) AS by_none,
       AVG(confidence) AS avg_confidence, COUNT(confidence) AS confidence_samples
  FROM sessions WHERE UNIX_TIMESTAMP(created_at) >= ? AND UNIX_TIMESTAMP(created_at) < ?
-- graphCounts :150-159   อาการยอดนิยม (จัดอันดับใน service)
SELECT graph_id, COUNT(*) AS n FROM sessions
 WHERE UNIX_TIMESTAMP(created_at) >= ? AND UNIX_TIMESTAMP(created_at) < ? GROUP BY graph_id
-- completedEndNodes :161-175   ที่ไหนจบกี่ครั้ง (service แปลงเป็นชนิดผลลัพธ์จากผังปัจจุบัน)
SELECT graph_id, current_node_id, COUNT(*) AS n FROM sessions
 WHERE status = 'completed' AND UNIX_TIMESTAMP(created_at) >= ? AND UNIX_TIMESTAMP(created_at) <
?
 GROUP BY graph_id, current_node_id
-- perDay :177-192   ยอดเริ่ม/จบรายวัน (25200 = 7 ชั่วโมง เวลาไทย)
SELECT FLOOR((UNIX_TIMESTAMP(created_at) + 25200) / 86400) AS day_idx,
       COUNT(*) AS started, SUM(status = 'completed') AS completed
  FROM sessions WHERE UNIX_TIMESTAMP(created_at) >= ? AND UNIX_TIMESTAMP(created_at) < ?
 GROUP BY day_idx
```

6. ตอบ:

```
{ "window": { "days": 30, "from": "2026-09-23", "to": "2026-10-22" },
  "sessions": { "total": 40, "completed": 25, "inProgress": 3, "expiredUnfinished": 12,
"byOwner": { "user": 10, "guest": 30, "none": 0 } },
  "feedback": { "total": 8, "helpful": 6, "notHelpful": 2, "satisfactionRate": 0.75 },
  "confidence": { "average": 0.79, "sampleSize": 20 },
  "outcomes": { "user_fixed": 15, "normal_behavior": 1, "handoff_informed": 8, "handoff_unknown":
1, "unknown": 0 },
  "topGraphs": [ { "graphId": "...", "entrySymptomTh": "...", "count": 7 } ], "topDevices": [ ...
], "perDay": [ { "date": "...", "started": 3, "completed": 2 } ] }
```

ข. รายการความเห็น `/admin/feedback`

1. `pages/AdminFeedbackPage.tsx` ตัวกรอง 👍/👎 + ช่วงวันที่ (`:50-92`) เก็บใน URL → `useAdminFeedback(params)` (`api/queries.ts:72`) → `api/admin.ts:12-20` **`GET /admin/feedback?helpful=&from=&to=&page=&pageSize=20`**
2. Backend: `admin.routes.ts:24-31` ด่าน `requireAuth` → `requireAdmin` → `parseFeedbackQuery` (`admin.dto.ts:122`): `helpful` ต้องเป็นข้อความ `'true'`/`'false'` · `from`/`to` เป็น `YYYY-MM-DD` เวลาไทย (ช่วงไม่เกิน 366 วัน, `from` ไม่หลัง `to`) · `page` ≥ 1 · `pageSize` 1–100 (ค่าเริ่มต้น 20)
3. `AdminService.feedback` (`admin.service.ts:30-49`) → `AdminStore.listFeedback` (`admin.store.ts:52-107`) ซึ่งต่อ `WHERE` จากตัวกรองที่ส่งมาเท่านั้น (ข้อความเงื่อนไขตายตัวในไฟล์ ค่าเข้าทาง `?`) แล้วยิง 2 คำสั่ง: `COUNT(*)` ทั้งหมด และรายการหน้านี้:

```
SELECT f.session_id, f.helpful, f.`comment`,
       UNIX_TIMESTAMP(f.created_at) AS created_epoch, UNIX_TIMESTAMP(f.updated_at) AS
updated_epoch,
       s.graph_id,
       CASE WHEN s.user_id IS NOT NULL THEN 'user' WHEN s.client_id IS NOT NULL THEN 'guest' ELSE
'none' END AS owner_type
  FROM session_feedback f JOIN sessions s ON s.session_id = f.session_id
 [WHERE f.helpful = ? AND UNIX_TIMESTAMP(f.created_at) >= ? AND ... < ?]
 ORDER BY f.created_at DESC, f.session_id
 LIMIT ? OFFSET ?
```

4. **ความเป็นส่วนตัว:** ไม่แตะตาราง `users` ไม่ SELECT อีเมล/ชื่อ/user_id/client_id รู้เจ้าของแค่ชนิด user/guest/none ผ่าน `CASE` และ service หยิบทีละช่อง ไม่ copy แถวจาก DB ทั้งก้อน (`admin.service.ts:33-46`)
5. หน้าจอแสดงแต่ละแถว (`FeedbackRow` `AdminFeedbackPage.tsx:139-167`): ป้าย 👍/👎, เวลา, ชนิดเจ้าของ, ข้อความ ("(ไม่มีข้อความ)" ถ้า `null`), ประเภทเครื่อง/ยี่ห้อ

ค. ฐานข้อมูลของช่วงนี้

- อ่าน `session_feedback` และ `sessions` (migrations `005_session_feedback.sql`, `004_sessions.sql`) · **ไม่แตะ `users`** · ไม่มีการเขียน

ข้อสังเกตช่วง 9

- feedback **ถูกใช้ในแดชบอร์ดจริง** (จำนวน 👍/👎 และอัตราความพอใจ) แต่ **ไม่ย้อนกลับไปปรับระบบ** (ไม่มีการเรียนรู้จาก feedback ในโค้ด)
- ตัวเลขนับจากตาราง `sessions` ซึ่งเอกสารเดิมบันทึกว่าเคย "ใช้ร่วมกับโปรเจกต์เก่า" (ผู้พัฒนายืนยันตอนล้าง DB ว่าฐานข้อมูลเก่าไม่ใช้แล้ว ข้อจำกัดนี้อาจเลิกเป็นจริง ยังไม่ได้ตรวจคอมเมนต์ให้ตรงกัน)
- "ยกเลิก" นับไม่ได้ (ยกเลิก = ลบแถวทิ้ง) มีแต่ "เดินค้างแล้วหมดอายุ"
- `AdminRoute` ฝั่งหน้าจอเป็น UX อย่างเดียว ตัวบังคับจริง `requireAdmin` (แก้ role ใน localStorage ได้หน้า แต่ API ตอบ 403 และ `/auth/me` ตอนเปิดเว็บแก้ role กลับตามเซิร์ฟเวอร์)
- ไม่มี dashboard ฝั่งผู้ใช้ทั่วไป (ค้นแล้ว มีเฉพาะ `/admin/*`)

เฉลยคำถามช่วง 9

- **ผู้ใช้ธรรมดาเรียก `/admin/analytics/summary` ตรงๆ:** 403 `FORBIDDEN` จาก `requireAdmin` (`auth.middleware.ts:70-80`) ก่อนถึงขั้น parse query
- **ไม่มีใครให้คะแนนในช่วงที่เลือก อัตราพอใจเป็นเท่าไร:** `null` (แสดง "-") ไม่ใช่ 0 เพราะ `feedback.total === 0` (`admin.service.ts:123`) 0 กับ "ไม่มีข้อมูล" ต่างความหมาย
- **ทำไมยิง 5 query พร้อมกันด้วย `Promise.all`:** ไม่ขึ้นต่อกัน รอพร้อมกันเร็วกว่ารอทีละตัว และใช้ช่วงเวลา `window` เดียวกัน ตัวเลขจึงสอดคล้องกัน
- **ทำไม `to` ในตัวกรองวันที่ไม่รวมปลาย:** `toEpoch` = เที่ยงคืนเวลาไทยของวันถัดไป ใช้ `<` จึงได้ข้อมูลครบทั้งวัน `to`
- **admin เห็นอีเมลของผู้ให้ความเห็นไหม:** ไม่เห็น ไม่มี SELECT อีเมล/user_id รู้แค่ชนิด user/guest/none

โน้ตของฉัน (ช่วง 9)

-

ภาพรวมไฟล์ที่แตะในช่วง 6–9 (เรียงตามลำดับการเรียก)

- **ช่วง 6:** `SessionPage.tsx` → `OutcomeForm.tsx` / `SessionHeader.tsx` → `api/queries.ts` → `api/traversal.ts` → `traversal.routes.ts` → `traversal.dto.ts` → `traversal.service.ts` → `session.store.ts` → `migrations/004_sessions.sql`
- **ช่วง 7:** `AppShell.tsx` → `HistoryPage.tsx` / `HistoryRow.tsx` / `HistoryDetailPage.tsx` → `api/queries.ts` → `api/history.ts` → `lib/clientHeaders.ts` → `history.routes.ts` → `auth/session-owner.ts` → `history.dto.ts` → `history.service.ts` → `history.store.ts` → `migrations/004_sessions.sql`, `005_session_feedback.sql`
- **ช่วง 8:** `HistoryDetailPage.tsx` → `FeedbackButtons.tsx` → `lib/textInput.ts` → `api/queries.ts` → `api/history.ts` → `history.routes.ts` → `history.dto.ts` → `history.service.ts` → `history.store.ts` → `migrations/005_session_feedback.sql`
- **ช่วง 9:** `AppShell.tsx` → `AdminRoute.tsx` → `AdminAnalyticsPage.tsx` / `AdminFeedbackPage.tsx` → `api/queries.ts` → `api/admin.ts` → `admin.routes.ts` → `auth/auth.middleware.ts` → `admin.dto.ts` → `admin.service.ts` → `admin.store.ts` → `migrations/004_sessions.sql`, `005_session_feedback.sql`